Bash for Bioinformatics

Ted Laderas

2025-10-02

Table of contents

Bash for Bio **3**
Who this course is for . . . 3
Learning Objectives . . . 3
Prerequisites . . . 4
Instructors / TAs . . . 4
Introductions . . . 4
Culture of the course . . . 5
Schedule . . . 5
Reference Texts . . . 6
Badge of completion . . . 7
First Survey . . . 8
Weekly Check in . . . 8
Acknowledgements . . . 8

1 Prework **9**
1.1 Navigating the Filesystems . . . 9
1.1.1 `pwd` Where Am I? . . . 9
1.2 Going `/home`: `~/` . . . 9
1.3 Absolute versus relative paths . . . 10
1.4 File Permissions . . . 11
1.5 Moving Things Around . . . 14
1.5.1 Keep Everything in Folders . . . 15
1.6 What's in the script . . . 15
1.7 Editing on a Linux Machine . . . 15
1.7.1 Try it Out . . . 16
1.7.2 Editing on your own machine . . . 16

2 Week 1: Navigating **17**
2.1 Learning Objectives . . . 17
2.2 Exercises . . . 17
2.3 Navigating the Bash Terminal . . . 17
2.4 Setting Yourself Up for Success . . . 18
2.5 Grabbing Stuff from GitHub . . . 18
2.5.1 `du`: How much space? . . . 19

2.6 FH users: the main filesystems . . . 20
2.6.1 More about the FH Filesystems . . . 21
2.6.2 Try it out . . . 21
2.7 Running a Bash Script . . . 22
2.8 Running an R or Python Script on the command line . . . 22
2.8.1 Loading the `fhR` or `fhPython` modules . . . 22
2.8.2 R Users . . . 23
2.8.3 Python Users . . . 23
2.9 Getting Help . . . 23
2.10 Recap . . . 24

3 Week 1: Introduction to Scripting 26
3.1 Exercises . . . 26
3.2 What we're working towards . . . 26
3.3 Motivation . . . 26
3.4 The first line: the she-bang . . . 27
3.5 Software Modules . . . 27
3.5.1 Is my software already installed? . . . 28
3.5.2 Environment Modules . . . 28
3.5.3 `module avail` . . . 29
3.5.4 `module load` . . . 29
3.5.5 Tip: Load only as many modules as you need at a time . . . 30
3.5.6 For more information . . . 30
3.6 `$1`: A Positional argument . . . 30
3.6.1 Positional Arguments such as `$1` . . . 31
3.6.2 For More Info . . . 31
3.7 `module purge` . . . 32
3.8 `shellcheck` . . . 32
3.9 Variables in Bash Scripts . . . 33
3.9.1 For more information . . . 34
3.10 Putting it all together . . . 34
3.10.1 Try it Out . . . 34
3.11 Aligning a file . . . 35
3.11.1 Try it Out . . . 36
3.12 Arguments in R Scripts . . . 36
3.12.1 Using Named Arguments in an R script . . . 36
3.12.2 Try it out: Running our R Script . . . 37
3.12.3 Integrating an R script into a Bash Script . . . 37
3.12.4 Summary . . . 37
3.13 Arguments in Python Scripts . . . 38
3.13.1 Using `sys.argv` . . . 38
3.13.2 Running our Python script on the command line . . . 38
3.14 Next Time . . . 39

4 Week 1 Exercises **40**
4.1 In class code examples . . . 40
4.1.1 Navigation and Running Scripts . . . 40
4.1.2 Scripting . . . 41
4.2 Homework Exercises . . . 42
4.2.1 Navigation and Running Scripts . . . 42
4.2.2 Scripting Exercises . . . 42

5 Week 2 Reading: HPC Basics **45**
5.1 Learning Objectives . . . 45
5.2 Important Terminology . . . 45
5.3 Understanding the key players . . . 46
5.3.1 Key Players in Local Computing . . . 46
5.3.2 Key Players in HPC . . . 47
5.3.3 Further Reading . . . 48
5.4 Sequence of Events of Running a Job . . . 48
5.4.1 Key Differences with local computing . . . 50
5.5 The Shared Filesystem . . . 50
5.5.1 Further Reading . . . 51
5.6 Requesting Machines . . . 51
5.6.1 Scattering: Distribute the Work . . . 51
5.7 What's Next? . . . 52

6 Week 2: Batch Processing and Submitting Jobs **53**
6.1 Exercises . . . 53
6.2 Learning Objectives . . . 53
6.3 Using `for` loops to cycle through files . . . 53
6.4 globs: selecting multiple files . . . 54
6.4.1 Try it out . . . 55
6.4.2 For more info on globs . . . 55
6.5 Batching on HPC . . . 56
6.5.1 SLURM Scripts . . . 56
6.5.2 SLURM Directives . . . 56
6.5.3 Job Arrays . . . 57
6.5.4 Try it out . . . 58
6.5.5 Processing lists of files using Job Arrays . . . 59
6.5.6 `scancel`ing a jobarray . . . 61
6.6 What's Next? . . . 61

7 Week 2 Exercises **62**
7.1 In-class . . . 62
7.2 Homework exercises . . . 62

8 Week 3 Reading: Containers **64**
8.1 Containers . . . 64
8.1.1 What is a Container? . . . 64
8.1.2 Docker vs. Apptainer . . . 64

9 Week 3: Containers **66**
9.1 Containers . . . 66
9.1.1 What is a Container? . . . 66
9.1.2 Docker vs. Apptainer . . . 66
9.2 Pulling a Docker Container and Running It . . . 67
9.2.1 Using a Container . . . 68
9.2.2 Bind Paths . . . 68
9.3 Testing code in a container . . . 69
9.3.1 Pulling a Docker Container . . . 69
9.3.2 Opening a Shell in a Container with `apptainer shell` . . . 69
9.3.3 Using bind paths in containers . . . 70
9.3.4 Executing in the Apptainer Shell . . . 72
9.3.5 Exiting the container when you're done . . . 72
9.3.6 Testing outside of the container . . . 72
9.3.7 The WILDS Docker Library . . . 73
9.4 What's Next? . . . 73

10 Week 3: Assignment **74**
10.1 In-class . . . 74
10.2 Homework . . . 75

11 Week 4 Reading: Pipes/JSON **77**
11.1 Using pipes: STDIN, STDOUT, STDERR . . . 77
11.1.1 > (redirects) . . . 78
11.1.2 For more info about pipes and pipelines . . . 79
11.2 What is JSON? . . . 79
11.2.1 How is JSON used? . . . 79
11.2.2 Elements of a JSON file . . . 79
11.2.3 JSON Input Files . . . 80

12 Week 4: Workflows **82**
12.1 Learning Objectives . . . 82
12.2 Getting today's exercises . . . 82
12.3 Running Workflows . . . 82
12.3.1 Specifying File Inputs . . . 83
12.3.2 Setting up for running a WDL workflow . . . 85
12.3.3 Running a WDL workflow . . . 85
12.3.4 Exploring the WDL output . . . 86

12.4 Looking at a WDL workflow . . . 87
12.4.1 How does this connect to shell scripting? . . . 89
12.5 `scatter()`: how we divide up tasks. . . . 90
12.5.1 More about `scatter` . . . 92
12.5.2 For more information . . . 93
12.6 Working with file manifests in WDL . . . 93
12.7 Where Next? . . . 94

13 Week 4 Exercises **95**
13.1 In class exercises: . . . 95
13.2 Homework Exercises . . . 95

14 Testing Scripts **96**
14.1 The cardinal rule of testing scripts . . . 96
14.1.1 Profiling Jobs using SLURM . . . 96
14.2 Developing and Testing scripts interactively . . . 97
14.2.1 Testing code on a compute node . . . 97
14.2.2 Grabbing an interactive shell on a worker . . . 98

15 Appendix: Configuring your Shell **100**
15.1 `.bashrc`: Where do I put my configuration? . . . 100
15.2 Aliases . . . 100
15.3 Environment Variables . . . 100
15.3.1 Setting Environment Variables . . . 101
15.3.2 A Very Special Environment Variable: `$PATH` . . . 102
15.3.3 Making your own environment variables . . . 103

16 Miscellaneous **104**
16.1 `hostname` What Machine am I on? . . . 104
16.1.1 Try it out . . . 104
16.2 Workflows . . . 104
16.2.1 One Workflow: `/fh/fast/` and `/hpc/temp/` . . . 104
16.2.2 Another Approach . . . 105
16.3 Quoting and Escaping Filenames in Bash . . . 106
16.3.1 If you have spaces in a filename, use double quotes (Most common) . . . 106
16.3.2 If you have a single quote in the name, use double quotes to wrap it (less common) . . . 106
16.3.3 Only escape characters when necessary (less common) . . . 106
16.3.4 If you need to preserve an escaped character, use single quotes (least common) . . . 107
16.3.5 For More Info . . . 107
16.4 Using pipes: STDIN, STDOUT, STDERR . . . 108
16.4.1 For more info about pipes and pipelines . . . 109

16.5 `basename` can be very handy when on workers 109
16.6 Advanced Batch Processing: Iterating using `xargs` 109
16.6.1 The Basic `xargs` pattern 111
16.6.2 For more information 112
16.7 `screen` or `tmux`: keep your session open 112
16.8 Using file manifests 114
16.9 Removing file extensions with variable expansion 116
16.10 Project/folder based workflows 116
16.10.1 Let's start with WDL Tasks 118
16.10.2 Anatomy of a Task 118
16.10.3 Architecture of a WDL file 119

Bash for Bio

Who this course is for

- Have you needed to align a folder of FASTA files and not know how to do it?
- Do you want to automate an R or Python script you wrote to work on a bunch of files?
- Do you want to do all of this on a high performance cluster ()?

If so, this course is for you! We will learn enough bash scripting to do useful things on the Fred Hutch computing cluster (affectionately called "gizmo") and automate the boring parts.

Learning Objectives

- **Apply** bash scripting to execute bioinformatics software, and Python/R scripts
- **Navigate** and **process** data on the different filesystems available at FH
- **Manage** software dependencies reproducibly using container-based technologies such as Docker/Apptainer containers or EasyBuild modules
- **Articulate** basic HPC architecture concepts and why they're useful in your work
- **Leverage** bash scripting to **execute** on a high performance cluster.
- **Utilize** workflow managers such as `cromwell/miniWDL` to process multiple files in a multi-step workflow.

> **Wasn't there another Bash for Bioinformatics book?**
>
> I originally wrote a book that was called Bash for Bioinformatics, which was about learning enough bash to use the cloud-based DNANexus platform effectively.
> I have renamed that book Bash for DNANexus, and named this course *Bash for Bioinformatics*.
> This book shares bones with *Bash for DNANexus*, but has more of a focus on running tasks on high performance computing systems ().

Prerequisites

- You will need an account on `rhino` and know how to connect to it through VPN. If you have taken the Intro to Fred Hutch Cluster Computing workshop, then you will be ready.
- Basic knowledge of the following commands:
  - `ls`
  - `cd` and basic directory navigation
  - `mv`/`cp`/`mkdir`/`rm`
- We highly recommend reviewing Intro to Command Line and Intro to Fred Hutch Cluster Computing. A list of the concepts you need to know are in the prework section.

We will assume that you will do all of your work in your home directory on Rhino. We will not be using that much space in your home directory.

> **Terminology**
>
> We know that not all of us have the same vocabulary. We try to define terminology as much as possible. These are indicated by double underlines such as this:
> You can click and hold on the term to define it.

Instructors / TAs

If you need to schedule some time to talk, please schedule with Ted.

- Ted Laderas (Main Instructor), Director of Training and Community, Office of the Chief Data Officer
- Chris Lo (TA), Data Science Trainer, Office of the Chief Data Officer
- Dan Tenenbaum (TA), HPC System Engineer III, Scientific Computing (Weeks 2-4)
- Taylor Firman (TA), Research Informatics Lead, Office of the Chief Data Officer (Weeks 3-4)

We all have experience running jobs on HPC and `gizmo`. Please reach out if you have any questions.

Introductions

In chat, please introduce yourself:

- Your Name & Your Group
- What you want to learn in this course

- Favorite spring activity or spring vegetable

Culture of the course

It is hard work teaching an online/hybrid course. If you can, please turn your camera on - it is difficult to teach to a group of blank screens.

- Learning on the job is challenging
  - I will move at learner's pace; we are learning together.
  - Teach not for mastery, but teach for empowerment to learn effectively.

We sometimes struggle with our data science in isolation, unaware that someone two doors down from us has gone through the same struggle.

- *We learn and work better with our peers.*
- *Know that if you have a question, other people will have it.*
- *Asking questions is our way of taking care of others.*

We ask you to follow Participation Guidelines and Code of Conduct.

Please note that this is the first time this course has been given - we have done our best to edit all mistakes out there, but there may be mistakes. So be patient and reach out if something isn't working.

If you do find a mistake, please report it to Ted. I'll add you to the acknowledgements below.

Schedule

Class is on Fridays, 12:00 - 1:30 PM. There will be an office hour 1/2 hour after class if you need help.

You should complete the readings before class, so we can hit the ground running.

Week	Date	Topics	Reading
Preclass		Review Intro to Command Line and Cluster 101	Prework
Week 1	May 15	Filesystem Basics and Writing and Running Bash Scripts	Bite Size Bash
Week 2	May 22	Batch Processing and HPC Jobs	HPC Basics

Week	Date	Topics	Reading
Week 3	May 29	Containers	Container Concepts
Week 4	June 5	Workflow Managers	Container Basics
On your own time	Testing Scripts		
On your own time	Configuring your Bash Shell		

Reference Texts

- We will be using Julia Evan's Bite Size Bash as our reference text. Julia's explanations are incredibly clear and it will be a valuable reference even beyond this course. The PDF is available in the Google Classroom materials. Please do not share with others - we have a group rate and it is only $12 for individual purchases.
- If you want to know the true power of the command line, I recommend Data Science at the Command Line. This book showcases how much you can get done with just command line.

Badge of completion

We offer a badge of completion when you finish the course!

What it is:

- A display of what you accomplished in the course, shareable in your professional networks such as LinkedIn, similar to online education services such as Coursera. A way for you to be accountable for your learning.

What it isn't:

- Accreditation through an university or degree-granting program.

Requirements:

- Sign up on the badging spreadsheet (will send link out in class).
- Complete badge-required sections of the exercises for 3 out of 4 assignments. We'll cover this in class.

First Survey

Please take the survey, as it helps me understand your background and how to tailor the course.

First Survey Link

Weekly Check in

If the topic is moving too quickly, or there is something else you want to let us know, please fill out the form. All responses are anonymous unless you want to know who you are.

https://forms.gle/yVgxiUpbeTx8sp5z8

Acknowledgements

This course would not be live without the efforts of:

- Emma Bishop
- Scott Chamberlain
- Taylor Firman
- Chris Lo
- Sonu Mishra
- Sitapriya Moorthi
- Dan Tenenbaum

Thanking you for all your help testing and editing this course.

1 Prework

The following is a review of some concepts you need to be familiar with before we start:

1.1 Navigating the Filesystems

1.1.1 `pwd` Where Am I?

The `pwd` command (short for *present working directory*) will let you know your current location in the filesystem. Knowing your current directory is critical when using *relative* file paths.

If I run `pwd` right after signing into `rhino` I get:

```
/home/tladera2
```

You should have a similar path, except with your user name. This is your *home directory* - where you have a limited amount of space (100 Gb) to store scripts and other files.

Don't worry, the majority of your data is stored elsewhere (see Section 2.6).

1.2 Going `/home`: `~/`

There is one important file alias you should always remember: `~/` is shorthand for your own *home directory*.

Depending on the linux distribution, this can be a different location. On the FH filesystem, when I use `~/`, it maps to:

```
/home/tladera2/
```

The home directory is also important because it is where your configuration files live, such as `.bashrc` (see Section 15.1).

Depending on how you work, you may want to store your scripts and workflows in `/home/`. Some people prefer to keep their scripts, data, and results in a single folder. This is not really practical for most genomics projects, unless you are saving processed data. For more info, see Section 16.10.

Your current working directory: .

There is an alias for your current directory: `.` (the period sign).
This becomes useful when you want to output files to your current location. We'll use this later.

1.3 Absolute versus relative paths

You may have muddled with file paths, and maybe have used absolute paths to specify the location of a file. When you are processing files, it is important to understand the difference.

Absolute paths contain all the information needed to find a file in a file system from the root `/` directory. For example, this would be an absolute path:

```
/fh/fast/laderast/immuno_project/raw_data/chr2.fa.gz
```

Absolute paths always start with `/`, because that is the root directory, where all the top folders and files live.

In terms of folder structure, this is what this looks like:

```
/                                                                    ①
  fh                                                                 ②
    fast
        laderast
|             immuno_project
                  raw_data
                     chr2.fa.gz
```

① Root directory
② Folders in root directory

Relative paths break up an absolute path into two pieces of information: 1) your current directory and 2) the path *relative* to that directory. Relative paths are really helpful because things don't break when you move your folder or files.

If my current working directory is the directory `/fh/fast/laderas_t/immuno_project/`, then the relative path to that same file would be:

```
raw_data/chr2.fa.gz
```

We can visualize the relative path like this, where our working directory is indicated by a star:

```
/                                                                    ①
  fh/fast/laderast/immuno_project/                                   ②
|                                    raw_data                        ③
                                       chr2.fa.gz
```

① The root directory
② Our working directory
③ Our relative path

Note that this relative path does not start with a `/`, because our current directory isn't the root directory. Relative paths are incredibly useful when scripting in a reproducible manner, such as using project-based workflows to process files in a single folder.

> **`<TAB>` is for autocompletion of paths**
>
> Never underestimate the usefulness of the `<TAB>` key, which triggers autocompletion on the command line. It can help you complete paths to files and save you a lot of typing. For example, say I have a path that I want to navigate to
> `/home/tladera2/my_long_path`
> I can type in the first part of the path and then hit `<TAB>`:
> `/home/tladera2/my_<TAB>`
> And if the prefix `my_` is unique in my folder, it will autocomplete the path:
> `/home/tladera2/my_long_path`
> Note that we need to use enough of the folder name so that completing it is unambiguous. If there are multiple choices, then autocomplete will list all of them.

1.4 File Permissions

File permissions are that are attached to file objects. They are how the system prevents certain files from being modified or restricting access of these files to certain people or groups.

All files have the following level of access permissions:

Level	Description
Owner-level	The owner of the file
Group-level	The group of the file
Everyone	The rest of the world

For example, if I'm the owner of the file, I can restrict the type of access to only myself (owner-level), the group I'm in (Group-level), or make the file freely available to everyone on the system (Everyone).

Each level has the following type of access:

Type	Description	Abbreviation	Example
Read	Level can only read contents of file	`r`	A list of users in a text file
Write	Level can write to the file	`w`	Appending an entry to the end of a log
Execute	Level can run the file as an executable	`x`	samtools

You can see the permissions for a file using the `ls -l <FILENAME>`. For example:

```
ls -l -h .
```

will give me the following line:

```
-rw-rw---- 1 tladera2 g_tladera2 1.8K Oct  2 15:21 01_assignment.qmd
-rw-rw---- 1 tladera2 g_tladera2  21K Oct  2 15:21 01_basics.qmd
-rw-rw---- 1 tladera2 g_tladera2 2.3K Oct  2 15:21 02_assignment.qmd
-rw-rw---- 1 tladera2 g_tladera2  21K Oct  2 15:21 02_scripting.qmd
-rw-rw---- 1 tladera2 g_tladera2 1.2K Oct  2 15:21 03_assignment.qmd
-rw-rw---- 1 tladera2 g_tladera2  13K Oct  2 15:21 03_batch.qmd
-rw-rw---- 1 tladera2 g_tladera2    0 Oct  2 15:21 04_assignment.qmd
```

It's this first part that we want to examine:

```
folder  everyone_else
|       |
-rw-rw----
 |  |
 you group
```

The 2nd to 4th letters define your own access to the files. The

```
rw-
```

Means that you have **r**ead and **w**rite access. If we had the following:

```
rwx
```

That would mean we have read, write, and e**x**ecute access.

Similarly, the 5th to 7th letters are the permissions for your group. In many cases at Fred Hutch, the group level maps to your lab or group.

The last 3 letters correspond to *everyone else* on the system. Think about whether you want to make your scripts accessible to everyone.

In this case, since the permissions are:

```
---
```

That means that everyone else cannot read, write, or execute your files. Your files are locked down.

The cardinal rule to remember is that:

> If you want to run a file as an executable, you (or your group) needs to have *executable level permission*.

For example, if I want to run a script called `run_samtools.sh` in my directory like this:

```
./run_samtools.sh my_bam_file.bam
```

I will need to have execute privileges at the user, group, or others level for `run_samtools.sh`.

We can change the permissions of our files using the `chmod` command.

Helpful unix permissions situations

I tend to just go by memory when setting file permissions. These are the common situations that I've encountered that I use regularly:

Situation	Command
Only I can execute/read/write a file	`chmod 700 <filename>`
Only I and my group can read a file	`chmod 110 <filename>`
Only I and my group can read/write/execute a file	`chmod 770 <filename>`
Grant my group read permissions	`chmod 710 <filename>`

Make executable/read/write by all `chmod 777 <filename>`

For right now, let's set scripts to be read/write/executable by us:

```
chmod -R 700 scripts/
```

The `-R` in this case is the `recursive` argument, which means apply `chmod` to everything within the `scripts/` folder.
This will save us some headaches when we try to execute our scripts.
If you have different permission situations that you need to set, please refer to: https://www.redhat.com/en/blog/linux-file-permissions-explained

1.5 Moving Things Around

A lot of the time, we need to move files between shared filesystems. One filesystem might be good at storage and be backed up on a regular basis, while another filesystem might be better for temporary work on the cluster.

You might be familiar with `mv`, which lets you move files around in Unix. One thing to keep in mind when you're `mv`ing things to a new folder that there is a difference between:

```
mv log.txt my_folder   ## renames log.txt to my_folder
```

and

```
mv log.txt my_folder/  ## moves log.txt to be in my_folder
```

This is one thing that still trips me up all the time.

This is one situation where using a GUI such as Motuz can be very helpful. You don't have to worry about accidentally renaming files.

Other tools for sync'ing between filesystems include `rsync`, which requires careful reading of documentation.

Things I always forget: the difference between `/home/mydir/` and `home/mydir/`

Some things that trip me up all the time. The difference between

```
/home/mydir/    #absolute path
```

and

```
home/mydir/     #relative path
```

The first one is an *absolute path*, and the second is a *relative path*. Your clue is the leading `/` at the beginning of a path. If you're getting `file not found` messages, check to make sure the path is the right format.

1.5.1 Keep Everything in Folders

We need to talk about code and data organization. For the FH system, we have a `/home/` directory, and if we have generated research data, a `/fh/fast/` directory. If we want our scripts to live in `/home/` and our data is in `/hpc/temp/`, we'll need to refer to each of these file locations.

Ideally, we want to make the naming conventions of our code and our data as similar as possible.

> **Try it Out**
>
> Copy the script `tell_the_time.sh` in the `scripts/week1/` directory to the top directory of `bash_for_bio`.
> Make the script executable.

1.6 What's in the script

We can see what's in the script by using `cat`:

```
cat tell_the_time.sh
```

And you'll get the following:

```
#!/bin/bash
date
```

1.7 Editing on a Linux Machine

On the `rhino` machines, we have the option to use the `nano` editor. `nano` is the most like a word processor or code editors.

- Open a file in `nano`: `nano <filename>`
- Save and quit: `<CTRL> + x` and then `yes`
- Navigate in file: using the arrow keys will work
- Find in file: `<CTRL> + w`
- Copy from outside the terminal (dependent on terminal program)

Pasting will depend on your terminal program. On macs it is Command-V, and on PuTTY it is using Right Click on the mouse

1.7.1 Try it Out

Try making your own file called `my_file.txt`:

```
nano my_file.txt
```

Add some text to it.

Use CTRL-X to exit, and make sure to select "Yes" to save.

1.7.2 Editing on your own machine

One thing you can do to make your life easier is to edit scripts with an editor on your own machine that is connected to `rhino` through Samba. You will able to open up files you create in the editor of you choice on your system, and when you save the file, the changes will be make on `rhino`.

Instructions are here: https://sciwiki.fredhutch.org/scicomputing/store_posix/#how-to-access-fred-hutch-storage and follow the instructions for mapping your network drive (Mac) and (Windows).

You will want to use the connection strings specified for `home`.

2 Week 1: Navigating

2.1 Learning Objectives

By the end of this session, you should be able to:

- **Navigate** and **copy** data to the different filesystems available at Fred Hutch.
- **Explain** the difference between *absolute* and *relative* file paths.
- **Set** Permissions on and **execute** a bash script
- **Execute** scripts written in Python and R on the command line
- **Find** help on the system and on the web

2.2 Exercises

Open up the exercises here or in Google Classroom.

> **Reminder about Terminology**
>
> Defined words are double underlined. You can *click and hold* on them to see the definition. Try it below!

2.3 Navigating the Bash Terminal

> We recommend that you review the material for Intro to Command Line and know the following: Changing directories and relative paths.

By default, when you log into a remote system such as `rhino`, you are in a .

Why is it a bash shell? Bash is the default shell for linux systems, especially for high performance clusters (HPCs), and there are some quirks about navigating around the command line you should be aware of.

> **A helpful key: `<Up Arrow>`**
>
> The key will let you cycle through your history, or previous executed commands. This

> can be super helpful if you have typed a long command with a syntax error. You can use `<Up Arrow>` to fix mistakes and run that command again.

2.4 Setting Yourself Up for Success

Make Sure you:

- ☒ Connect to VPN
- ☒ Keep the Bash for Bioformatics page open
- ☒ Open the assignment in Google Classroom to take notes
- ☒ Connect to `rhino` using `terminal` or `PuTTY` or `On Demand` (see below).

I will demo how to connect to `rhino` using the Scicomp `On Demand` dashboard. This site has a handy "Rhino Shell Access" menu item under "Clusters".

When you are scripting, I suggest you open two terminal windows: the first one is for editing scripts, and the second one is for running scripts on the command line.

So now we have logged into `rhino`. Now what?

2.5 Grabbing Stuff from GitHub

For the rest of the exercises for today, we'll be grabbing the scripts from github using `git clone`.

```
git clone https://github.com/fhdsl/bash_for_bio
```

This will create a folder called `bash_for_bio/` in our current directory. This directory has all of the course materials, including the scripts.

> ❗ Stay in `bash_for_bio/`
>
> Throughout this course, I expect you to run code in the base `bash_for_bio/` folder, not in `scripts` or in `data`. All of the code is tested with this in mind.
> If you are having problems executing the code, please make sure you are in the base `bash_for_bio` folder, or adjust your file paths when running the script.

2.5.1 `du`: How much space?

One of the first things we can do is check for disk usage with the `du` command. If I run `du` by itself on the command line, it will give me the disk usage of all folders and files in our current directory, which is a lot of output.

There is an option called `-d` that lets us specify the *depth*. `-d 1` will give us only the file sizes of the top level folders in our directory.

Make sure you are in the `bash_for_bio/` directory. Then try the following command:

```
du -d 1 -h .
```

Here are the first few lines of my `du` output within the `bash_for_bio` folder:

```
240K    ./_extensions
192K    ./.quarto
616K    ./scripts
1.9M    ./data
8.6M    ./.git
6.7M    ./docs
10M ./images
30M .
```

If we want to specify `du` to scan only a single folder, we can give the folder name.

```
du -d 1 scripts
```

And I will get the following output:

```
144K    scripts/week1
56K scripts/__pycache__
128K    scripts/week3
232K    scripts/week2
616K    scripts
```

> **Try it out**
>
> Try checking the disk usage using `du` for the `bash_for_bio/` folder in your `/home` directory (mine is `/home/tladera2/bash_for_bio/`).

```
du -d 1 bash_for_bio/
```

Try out using `du -d 2` on your home directory:

```
du -d 2 ~/
```

2.6 FH users: the main filesystems

When working on the Fred Hutch HPC, there are four main filesystems you should consider:

- `/home/` - The home filesystem. Your scripts can live here. Also where your configuration files (such as `.bashrc`) live. Can be accessed using `~/`.
- `/fh/fast/` (also known as `fast`) - Research storage. Raw files and processed results should live here.
- `/hpc/temp/` (also known as `temp`) - The temporary filesystem. This filesystem is faster to access for gizmo nodes on the cluster, so files can be copied to for computation. The output files you generate should be moved back into an appropriate folder on `/fh/fast/`. Note that files on `/hpc/temp/` will be deleted after 30 days.
- `/fh/regulated/` - A secure filesystem meant for NIH regulated data. If you are processing data that is regulated under the current NIH guidelines, you will process it here.

So, how do we utilize these filesystems? We will be running commands like this:

```
ml BWA                                                                  ①
bwa mem -M -t 2 \ # <2>
  /fh/fast/reference_data/chr20 \ # <3>
  /fh/fast/laderas_t/raw_data/na12878_1.fq \ # <4>
  /fh/fast/laderas_t/raw_data/na12878_2.fq > \ # <4>
  /hpc/temp/laderas_t/aligned_data/na12878_1.sam                        ⑤
```

① Load bwa software
② Start `bwa mem` (aligner)
③ path of genome index
④ path of paired end reads files
⑤ path of output

To understand the above, We first have to familiarize ourselves with *absolute* vs *relative* paths.

> **When you need to span multiple lines: **
>
> Sometimes it's hard to read code that is a single line. You can break up a very long line

of code using the \ (backslash) character.
For example, instead of:

```
bwa mem -M -t 2 /fh/fast/reference_data/chr20 /fh/fast/laderas_t/raw_data/na12878_1.fq /fh/
```

We can rewrite it as:

```
ml BWA
bwa mem -M -t 2 \
  /fh/fast/reference_data/chr20 \ # <1>
  /fh/fast/laderas_t/raw_data/na12878_1.fq \ # <2>
  /fh/fast/laderas_t/raw_data/na12878_2.fq > \ # <3>
  /hpc/temp/laderas_t/aligned_data/na12878_1.sam                    ④
```

① Path of reference genome
② 1st paired end read FASTQ file
③ 2nd paired end read FASTQ file
④ Output file location

We'll use this throughout the book so the code is easier to read.

2.6.1 More about the FH Filesystems

https://sciwiki.fredhutch.org/scicomputing/store_posix/

Even if you don't have execute permissions

With `bash` scripts, you can still run them if you have `read` permissions. You can still run bash scripts by using the `bash` command:

```
bash run_samtools.sh my_bam_file.bam
```

This is also the case for scripts that use she-bangs (Section 3.4) for R or Python or any other executable.

2.6.2 Try it out

What are the permissions for the GitHub repo (bash_for_bio) that you just downloaded?

2.7 Running a Bash Script

Ok, now we have a bash script `tell_the_time.sh` in our current directory, how do we run it?

Because the script is not on our `$PATH` (Section 15.3.2), we'll need to use `./` to execute it. `./` is an alias for the current folder, and it is an indicator to bash that the command we want to execute is in our current folder.

```
tladera2$ ./tell_the_time.sh
```

If we haven't set the permissions (Section 1.4) correctly, we'll get this message:

```
bash: ./scripts/tell_the_time.sh: Permission denied
```

But if we have execute access, we'll get something like this:

```
Fri Jul 11 13:27:47 PDT 2025
```

Which is the current date and time.

2.8 Running an R or Python Script on the command line

2.8.1 Loading the `fhR` or `fhPython` modules

Before we can run our scripts in R or Python, we'll need to load up either R or Python on the cluster. We can do this with the `module load` command:

```
module load fhR                                                    ①
module load fhPython                                               ②
```

① Load up `fhR` module - has R and most packages installed
② Load up `fhPython` module - has Python and most packages installed.

We'll talk more about software modules next week (Section 3.5).

2.8.2 R Users

You might not be aware that there are multiple ways to run R:

1) as an interactive console, which is what we usually use in an IDE such as RStudio
2) on the command line using the `Rscript` command.

```
Rscript my_r_script.R
```

To run this script, we'll need to first load `fhR`:

```
module load fhR
Rscript my_r_script.R
module purge
```

2.8.3 Python Users

Python users are much more aware that you can run Python scripts on the command line:

```
python3 my_python_script.py
```

To execute this on `gizmo`, we'll first need to load `fhPython`:

```
module load fhPython
python3 my_python_script.py
module purge
```

Within a shell script, you can also use a shebang (Section 3.4) to make your script executable by providing the location of `python3`:

```
#!/bin/python3
python3 my_python_script.py
```

2.9 Getting Help

You may have heard about man pages. You can usually get help by using the `man` command:

```
man wc
```

This is the first part of the output:

```
NAME
     wc - word, line, character, and byte count

SYNOPSIS
     wc [--libxo] [-Lclmw] [file ...]

DESCRIPTION
     The wc utility displays the number of lines, words, and bytes contained in each input fi
     specified) to the standard output.  A line is defined as a string of characters delimite
     Characters beyond the final  newline  character will not be included in the line count.

     A word is defined as a string of characters delimited by white space characters.  White
     characters for which the iswspace(3) function returns true.  If more than one input file
     counts for all the files is displayed on a separate line after the output for the last i

     The following options are available:

         --libxo
             Generate output via libxo(3) in a selection of different human and machine reada
             for details on command line arguments.

     -L      Write the length of the line containing the most bytes (default) or characters
m is provided) to standard
             output.  When more than one file argument is specified, the longest input line o
             of the final "total".
```

I personally find man pages very hard to read, especially when there are lots of options for a command.

Instead, I use `tldr`, which contain examples of the most commonly used options in a command. It is not installed on gizmo, but you can use the page at https://tldr.inbrowser.app/, which has all of the `tldr` help pages.

2.10 Recap

We learned the following this week:

- **Navigate** and **copy** data to the different filesystems available at Fred Hutch.
- **Set** Permissions on and **execute** a bash script
- **Find** help on the system

3 Week 1: Introduction to Scripting

3.1 Exercises

Open up the exercises here.

3.2 What we're working towards

By the end of this session, you should be able to understand and run this shell script.

```
#| filename: ./scripts/week2/sam_count.sh
#!/bin/bash                                                    ①
module load SAMtools/1.19.2-GCC-13.2.0                         ②
samtools view -c $1 > $1.counts.txt                            ③
module purge                                                   ④
```

① Specify the shell language (bash)
② load the module
③ run the script
④ purge the module.

It seems a little intimidating, but we will take this apart line by line.

3.3 Motivation

There is a rule in programming: if you do something more than 3 times, you should consider making it into a script or function.

For example, imagine that you use `samtools view -c` all the time with certain options and you want to save the output. You can put this command and options into a shell script that takes named files as an argument (such as `samcount.sh`. Instead of typing `samtools view -c` over and over again, you can run

```
./sam_count.sh my_file.bam
```

and it will output a file `my_file.bam.counts.txt` that has the counts we are interested in.

3.4 The first line: the she-bang

What's this first line?

```
#!/bin/bash
```

the `#!` is known as a she-bang - it's a signal to Linux what language interpreter to use when running the script on the command line. In our case, we want to use `bash`. But we can also make python scripts executable by putting the path to python:

```
#!/
```

The she-bang is necessary if you want to run the script without using the `bash` command (after you have made it executable):

```
./samcount.sh chr1.sam
```

Note that you can always use `bash` to execute a file if you don't have execute access:

```
bash samcount.sh chr1.sam
```

3.5 Software Modules

Ok, we've gotten comfortable navigating around the HPC filesystem. Now how do we run executables on files?

Let's talk about the two problems:

1) How do we find executables on a cluster, and
2) how do we load them up and run them?

3.5.1 Is my software already installed?

Say we want to see if `samtools` is installed on our HPC. One of the key commands you can use to find software is the `which` command. If your software is installed, `which` will give you the path where the software is installed. For example, I can see if `bash` is installed:

```
which bash
```

Which gives me the response:

```
/bin/bash
```

So, let's see if `samtools` is installed:

```
which samtools
```

Which gives no response, so where is `samtools`?

If we don't have `samtools` immediately available, how do we find it on our system? On the HPC system, We can use environment modules to load software.

3.5.2 Environment Modules

Before you install your own versions of software, it's important to realize that this problem may be solved for you.

Your first stop should be looking for environment modules on the HPC. Not all HPCs have these, but if they have them, this should be your first stop to find executables.

`lmod` is a system for loading and unloading software modules. It is usually installed on HPCs. The commands all start with `module`, and there are a number of ones that are useful for you.

- `module avail`
- `module load`
- `module purge`

If you want to see the current list of available modules and their names, check them out here.

Looking for `samtools` on that page, we discovered the name of our module:

```
SAMtools
```

So, that's what we'll use to load up `samtools`.

3.5.3 `module avail`

We can find the versions of `SAMtools` available to us by using:

```
module avail SAMtools
```

And we'll get a list of versions:

```
------------------------------- /app/modules/all -------------------------------
   SAMtools/1.10-GCC-8.3.0        SAMtools/1.16.1-GCC-11.3.0
   SAMtools/1.10-GCCcore-8.3.0    SAMtools/1.17-GCC-12.2.0
   SAMtools/1.11-GCC-10.2.0       SAMtools/1.18-GCC-12.3.0
   SAMtools/1.14-GCC-11.2.0       SAMtools/1.19.2-GCC-13.2.0
   SAMtools/1.15.1-GCC-11.2.0     SAMtools/1.21-GCC-13.3.0   (D)
   SAMtools/1.16.1-GCC-11.2.0
```

The `(D)` is the default version. It is the version we use if we just use:

```
module load SAMtools
```

Without a version. In general, it is good to fix the module version in your scripts for reproducibility reasons. The reason for that is that the default version may change in the future, and there may be changes that break your code.

3.5.4 `module load`

Here's the next line of the script:

```
module load SAMtools/1.19.2-GCC-13.2.0  #load the module
```

Our module name is `SAMtools`, and the `1.19.2-GCC-13.2.0` after it is the version of that module.

> **For FH Users: Modules benefit everyone**
>
> If there is a particular bit of software that you need to run on the FH cluster that's not there, make sure to request it from SciComp. Someone else probably needs it and so making it known so they can add it as a Environment module will help other people.

> **For FH Users**
>
> On the FH cluster, `ml` is a handy command that combines `module load` and `module avail`.
> You can load a module with `ml <module_name>`.

3.5.5 Tip: Load only as many modules as you need at a time

One of the big issues with bioinformatics software is that the toolchain (the software dependencies needed to run the software) can be different for different executables. This is also called dependency hell. So when possible, load only one or two modules at a time for each step of your analysis. When you're done with that step, use `module purge` to clear out the software environment.

3.5.6 For more information

Please refer to https://sciwiki.fredhutch.org/scicomputing/compute_environments/ for more information about the available modules.

3.6 `$1`: A Positional argument

The next line of our script is this:

```
samtools view -c $1 > $1.counts.txt
```

Let's take a look at the command that we're running first. We're going to run `samtools view -c`, which will give us counts on an incoming `bam` or `sam` file and save it in a file. We want to be able to run our script like this:

```
bash sam_count.sh my_file.bam
```

When we run it like that, `sam_count.sh` will run `samtools view -c` like this:

```
samtools view -c my_file.bam > my_file.bam.counts.txt
```

So what's going on here is that there is some substitution using common arguments. Let's look at these.

> - redirecting outputs to a file

The > in the script means that we are going to direct the *output* of `samtools view -c` into a file.
If we didn't do this, `samtools_count.sh` would output everything to console.
Much more info about this when we talk about the different outputs to console (Section 11.1).

3.6.1 Positional Arguments such as $1

How did the script know where to substitute each of our arguments? It has to do with the argument variables. Arguments (terms that follow our command) are indexed starting with the number 1. We can access the value at the first position using the special variable `$1`.

Note that this works even in quotes.

So, to unpack our script, we are substituting our first argument for the `$1`.

When we run this script, it will output into our *current* directory, which is the top level of the `bash_for_bio` folder. We can change the behavior of the script by doing the following:

```
samtools view -c $1 > $2
```

With the added complication that we need to specify a path for our counts file.

Test yourself

How would we rewrite `sam_run.sh` (shown below) if we wanted to specify the output file as the first argument and the bam file as the second argument?

```
#!/bin/bash/
samtools view -c $1 > $2
```

Answer

For this script, we would switch the positions of `$1` and `$2`.

```
#!/bin/bash/
samtools view -c $2 > $1
```

3.6.2 For More Info

Please refer to Bite Size Bash page 9 for more info.

3.7 `module purge`

The last line of our script is:

```
module purge
```

This line will unload the modules from memory. It's good practice to unload modules when you're done with them, especially since they have complex chains of dependencies, and the versions of these dependencies can interfere with other packages.

> **Processing Files Best Practices**
>
> One thing to remember is to not touch the raw data. The original files should remain untouched. That is, reading the raw data files is fine, but avoid overwriting the orignal data files.
> A good way to do this is to have your outputs saved in a different folder.

3.8 `shellcheck`

When you are writing shell scripts, `shellcheck` is a really handy way to check that you have syntax correctly.

```
#/bin/bash                                                    ①
echo "blah blah"
```

① A mistake! should be `#!/bin/bash`.

If we run `shellcheck` on this script, it will catch the mistake:

```
ml shellcheck   #load the shellcheck module
shellcheck scripts/week2/bad_script.sh
```

`shellcheck` will give the following response:

```
In bad_script.sh line 1:
#/bin/bash
 ^-- SC1113 (error): Use #!, not just #, for the shebang.
```

`shellcheck` is available on `rhino` via `ml`, you can also install the VSCode Extension if you are editing in VSCode.

For more info about `shellcheck`, refer to Bite Sized Bash, page 6.

> **When you have spaces in file names: escaping characters or double quotes**
>
> In general, I try not to use spaces in my file names. They can be unwieldy.
> The general solution is to use double quotes or an escape character `\`
> For example, if my file name is `CALU final.fasta`
> I can refer to it as:
>
> ```
> ./scripts/week2/run_bwa.sh "CALU final.fasta"
> ```
>
> The escape character (\) is a way to put spaces in a bash command without bash interpreting everything after the space as an argument. Essentially, it tells `bash` to literally interpret the character that follows it.
>
> ```
> ./scripts/week2/run_bwa.sh CALU\ final.fasta
> ```
>
> I prefer to use double quotes. You'll notice that all file names in this course use an underscore (`_`) or dashes (`-`) instead of spaces, which makes them easier to handle.

3.9 Variables in Bash Scripts

We saw a little bit about using `$1`, which is a *variable* in our Bash scripts. Let's talk about declaring variables in bash scripts and using them using variable expansion.

In Bash, we can declare a variable by using `<variable_name>=<value>`. Note there are no spaces between the variable (`my_variable`), equals sign, and the value (`"ggplot2"`).

```
my_variable="ggplot2"

echo "My favorite R package is ${my_variable}"
```

```
My favorite R package is ggplot2
```

Take a look at line 3 above. We expand the variable (that is, we substitute the actual variable) by using `${my_variable}` in our `echo` statement.

In general, when expanding a variable in a quoted string, it is better to use `${my_variable}` (the variable name in curly brackets). This is especially important when using the variable name as part of a string:

```
my_var="chr1"
echo "${my_var}_1.vcf.gz"
```

```
chr1_1.vcf.gz
```

If we didn't use the braces here, like this:

```
echo "$my_var_1.vcf.gz"
```

Bash would look for the variable `$my_var_1`, which doesn't exist. So use the curly braces `{}` when you expand variables. It's safer overall.

> **Subshells**
>
> There is an alternate method for variable expansion which we will use when we call a *sub-shell* - a shell within a shell. We need to use parentheses `()` to expand these commands within the sub-shell, but not the top-shell. We'll use this when we process multiple files within a single worker.

3.9.1 For more information

Please refer to Bite Size Bash page 7 for more information.

3.10 Putting it all together

We can run the script on a file (assuming we are in `bash_for_bio/`):

```
./scripts/week2/sam_count.sh ./data/my_file.bam
```

3.10.1 Try it Out

Try out the above script on `./data/CALU1_combined_final.sam`

Make sure you are in the top folder (`bash_for_bio`) when you do it.

> **When should you hard code a path?**
>
> In general, hard coding absolute paths (such as `/home/laderast/bash_for_bio/scripts/week1/run_this.sh`) makes your code less reproducible. Using a project-based workflow (where your paths mostly refer within a single folder) makes it more reproducible.
> Of course, there are cases (such as hard coding a genome reference) which should use absolute paths, since the location of these differs across systems. Just make a note when sharing which paths are hard-coded.
> I talk about the benefits of project based workflows here: (Section 16.10).

3.11 Aligning a file

How about running the `bwa-mem` aligner? Let's run a FASTQ file into it using `${1}`

```
#| filename: scripts/week2/run_bwa.sh
#!/bin/bash
module load BWA/0.7.17-GCCcore-11.2.0                                    ①
input_fastq=${1}                                                         ②
# strip path and suffix
base_file_name="${input_fastq%.fastq}"
base_file_name=${base_file_name##*/}
echo "running $input_fastq"
sample_name="SM:${base_file_name}"
read_group_id="ID:${base_file_name}"
platform_info="PL:Illumina"
ref_fasta_local="/shared/biodata/reference/iGenomes/Homo_sapiens/UCSC/hg19/Sequence/BWAIndex/

bwa mem \ # <3>
      -p -v 3 -M \
      -R "@RG\t${read_group_id}\t${sample_name}\t${platform_info}" \
      "${ref_fasta_local}" "${input_fastq}" > \
      "${base_file_name}.sam"
module purge
```

① Load `BWA` module. Note that we are using a specific module version.
② Set a variable called `$ref_fasta_local`, using the location of the `hg19` reference.
③ Run `bwa mem` on the `$input_fastq` file, using the `$ref_fasta_local` variable as the location of the reference genome, and output to `${base_file_name}.sam`.

3.11.1 Try it Out

We can run this script using:

```
./scripts/week2/run_bwa.sh ./data/CALU1_combined_final.fasta
```

Where does the output file end up?

3.12 Arguments in R Scripts

To make our R Script more useful, it must also accept arguments. Let's explore how to achieve this.

As we learned in (Section 2.8), We can use `Rscript` to execute our command *non-interactively*. `Rscript`, which is the command line executable, will run our Rs cript on the command line.

Note that we have a named argument called `input_file` and it is done differently than in Bash - how do we use this in our R Script?

3.12.1 Using Named Arguments in an R script

We can pass arguments from our bash script to our R script by using `commandArgs()` - this will populate a list of named arguments (such as `input_file`) that are passed into the R Script. We assign the output of `commandArgs()` into the `args` object.

We refer to our `input_file` argument as `args$input_file` in our script.

```
#| eval: false
#| filename: "scripts/week2/process_data.R"
library(tidyverse)

args <- commandArgs()

# Use arg$CSVFILE in read.csv
csv_file <- read.csv(file=args$input_file)                    ①
```

```r
# Do some work with csv_file
csv_filtered <- csv_file |> janitor::clean_names()

# Write output
write.csv(csv_filtered, file = paste0(args$CSVFILE, "_cleaned.csv"))        ②
```

① Use `args$input_file` as an input for `read.csv()`
② Use `args$input_file` to specify the output name.

3.12.2 Try it out: Running our R Script

Now that we've set it up, we can run the R script from the command line as follows:

```
#| eval: false
module load fhR
Rscript process_data.R input_file="LUSC_clinical.csv"
module purge
```

3.12.3 Integrating an R script into a Bash Script

Say you have an R Script you need to run on the command line in combination with other unix commands. In our bash script, we can do the following:

```
#| filename: "./scripts/week2/wrap_r_script.sh"
#| eval: false
#!/bin/bash
module load fhR
Rscript process_data.R input_file="${1}"
module purge
```

Then we can run

```
#| eval: false
bash wrap_r_script.sh .data/PRAD.csv
```

In our bash script, `my_bash_script.sh`, we're using a positional argument (for simplicity) to specify our csvfile, and then passing the positional argument to named ones (`input_file`) for `my_r_script.R`.

3.12.4 Summary

- R scripts will accept named arguments when called by `Rscript`:

```
Rscript process_csv.R input_file="my_csvfile.csv"
```

3.13 Arguments in Python Scripts

Similarly, we will need Python scripts to accept arguments as well.

3.13.1 Using `sys.argv`

The `sys` module (built into Python) will let us access arguments by position:

[argparse]

```
#| eval: false
#| filename: process_file.py
import sys                                                    ①
import pandas as pd
file = sys.argv[1]                                            ②
df = pd.read_csv(file)
```

① Import `sys` module
② Use `sys.argv` to access the first argument `sys.argv[1]`

If you know a little bit about Python, you'll know that arrays in python are *zero-indexed*, that is, they start at `[0]`, but we have started here at `sys.argv[1]` - what is at `sys.argv[0]`?

It turns out that `sys.argv[0]` is the name of the script. In the above script `process_file.py`, it contains the string `"process_file.py"`. Having the name of the script can come in handy, especially when outputting things.

3.13.2 Running our Python script on the command line

Now we have set up our python script, we can run it on the command line on `rhino`:

```
#| eval: false
module load fhPython
python3 process_file.py lusc_file.csv
module purge
```

And we could wrap it up in a bash script:

```
#| eval: false
#| filename: ./scripts/week2/wrap_py_script.sh
#!/bin/bash
module load fhPython
python3 process_file.py ${1}
module purge
```

3.14 Next Time

We'll work on batch processing, both on a local system and the HPC cluster. Make sure to read the HPC Basic chapter before class.

4 Week 1 Exercises

4.1 In class code examples

4.1.1 Navigation and Running Scripts

1. Clone the `bash_for_bio` repository into your home directory:

```
git clone https://github.com/fhdsl/bash_for_bio
```

Change directories:

```
cd bash_for_bio
```

2. Try checking the disk usage using `du` for the `bash_for_bio` folder in your `/home` directory (mine is `/home/tladera2/bash_for_bio`).

```
du -d 1 -h /home/tladera2/bash_for_bio/
```

Try out using `du -d -h 2` on your home directory:

```
du -d 2 ~/
```

3. What are the permissions for the GitHub repo (bash_for_bio) that you just downloaded?
4. Copy the script `tell_the_time.sh` in the `scripts/week1/` directory to the top directory of `bash_for_bio`. Make the script executable. Run the script.

```
#put code here
```

5. Try making your own file called `my_file.txt`:

```
nano my_file.txt
```

Add some text to it.

Use CTRL-X to exit, and make sure to select "Yes" to save.

6. Change permissions for the `scripts/` folder:

```
chmod -R 700 scripts/
```

4.1.2 Scripting

1. We can run the script on a file (assuming we are in `bash_for_bio/`):

```
./scripts/week1/sam_count.sh ./data/my_file.bam
```

Try out the above script on `./data/CALU1_combined_final.sam`.

Make sure you are in the top folder (`bash_for_bio`) when you do it.

2. Run `run_bwa.sh` using:

```
./scripts/week1/run_bwa.sh ./data/CALU1_combined_final.fasta
```

3. Run `process_data.R`:

```
module load fhR
Rscript process_data.R input_file="LUSC_clinical.csv"
module purge
```

4. Run `process_data.py`:

```
module load fhPython
python3 process_file.py lusc_file.csv
module purge
```

4.2 Homework Exercises

4.2.1 Navigation and Running Scripts

All exercises are required for the badge.

1. Copy a file from the `scripts/week1/` directory in `bash_for_bio/` to your own home directory.

`/home/tladera2/bash_for_bio/scripts/week1/run_this.sh`

Check the permissions. If necessary, change the script to read/write/executable for yourself:

`chmod --- run_this.sh`

Try running it in your home directory - did it work?

`./run_this.sh`

2. (pick python or R) Take a look at `scripts/week1/rnorm.R` or `scripts/week1/random_num.py`. Load up the `fhR` or `fhPython` modules on `rhino` using `module load`. Run it on the command line with `Rscript` or `python3`.

Did you need to make this script executable before you ran it?

```
module load _____
--------
module purge
```

4.2.2 Scripting Exercises

All exercises are required for the badge. Where possible, please paste your code in the grey boxes and output below that. (If the output is long, then just the first few lines is fine.) Make sure to answer the questions.

3. Copy the below script into a file called `samtools_count.sh`. What does the script do?

```
#!/bin/bash
module load SAMtools
samtools view -c $1
```

How would we modify the above script to redirect the output to `${1}.counts.txt`?

4. Make sure you can get `scripts/week1/run_bwa.sh` to run on `rhino`. Run it on `data/MOLM13_combined_final.fastq`.

When it's successful, run `head` on the `MOLM113_combined_final.fastq` output, and paste your command and the output below.

5. Modify `scripts/week1/run_bwa.sh` to
    a. Take an additional argument, a folder path
    b. Save the SAM file to this folder path

Hint 1: I recommend that you copy `run_bwa.sh` into a new script and work from there. Put your code into the codeblock below:

Write an example to run your new version of the script below and run it:

For question 6, pick one language to answer.

6R. (R) Modify the below R script and save it to a file called `scripts/week2/r_csv_script.R`. It should also take an argument called $FILEPATH for `read.csv()`:

```
library(tidyverse)
csv_file <- read_csv("myfile.csv")
summary(csv_file)
```

How would you run this on the command line?

```
module load fhR
-----
module purge
```

How would you redirect the output of your script to a file?

6Py. (py) Modify the below Python script to be runnable and save it to `scripts/week2/py_csv_script.py`. Your new version should also take the first position argument (a file path) and process the file:

```python
#| eval: false
import pandas as pd
csv_file = pd.read_csv("my_file.csv")
csv_file.describe()
```

How would you run this on the command line?

```
module load fhPython
-----
module purge
```

How would you redirect the output of your script to a file?

5 Week 2 Reading: HPC Basics

We all need to start somewhere when we work with High Performance Computing (HPC).

This chapter is a review of how HPC works and the basic concepts. If you haven't used HPC before, no worries! This chapter will get you up to speed.

5.1 Learning Objectives

After reading this chapter, you should be able to:

1. **Define** key players in both local computing and HPC
2. **Articulate** key differences between local computing and HPC
3. **Describe** the sequence of events in launching jobs in the HPC cluster
4. **Differentiate** local storage from shared storage and **articulate** the advantages of shared storage.
5. **Describe** the differences between a single job and a batch job.

5.2 Important Terminology

Let's establish the terminology we need to talk about HPC computing.

- **High Performance Computing** - A type of computing that uses higher spec machines, or multiple machines that are joined together in a *cluster*. These machines can either be on-premise (also called *on-prem*), or in the cloud (such as Amazon EC machines, or Azure Batch).
- **Cluster** - a group of machines networked such that users can use one or more machines at once.
- **Allocation** - a temporary set of one or more computers requested from a cluster.
- **Toolchain** - a piece of software and its dependencies needed to build a tool on a computer. For example, `cromwell` (a workflow runner), and `java`.
- **Software Environment** - everything needed to run a piece of software on a brand new computer. For example, this would include installing `tidyverse`, but also all of its dependencies (`R`) as well. A *toolchain* is similar, but might not contain all system dependencies.

- **Executable** - software that is available on the HPC.
- **Shared Filesystem** - Part of the HPC that stores our files and other objects. We'll see that these other objects include applets, databases, and other object types.
- **SLURM** - The workload manager of the HPC. Commands in **SLURM** such as `srun`, (see Section 5.4) kick off the processes on executing jobs on the **worker nodes**.
- **Interactive Analysis** - Any analysis that requires interactive input from the user. Using RStudio and JupyterLab are two examples of interactive analysis. As opposed to *non-interactive* analysis, which is done via scripts.

5.3 Understanding the key players

In order to understand what's going on with HPC, we will have to change our mental model of computing.

Let's contrast the key players in local computing with the key players in HPC.

5.3.1 Key Players in Local Computing

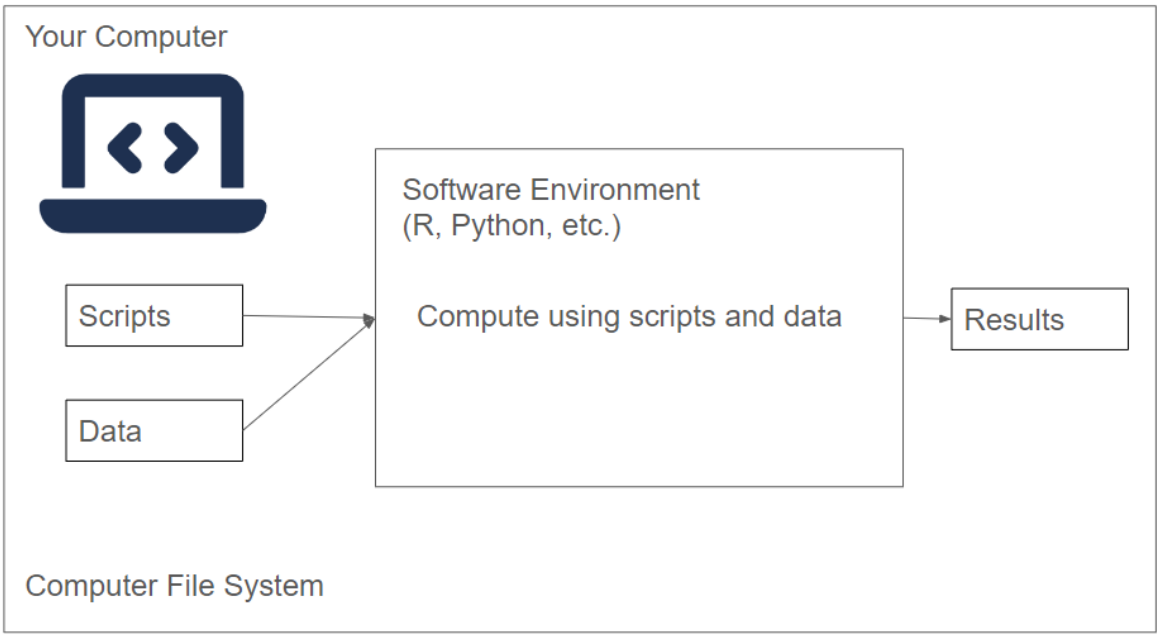


Figure 5.1: Local Computing

- **Our Machine**

When we run an analysis or process files on our computer, we are in control of all aspects of our computer. We are able to install a software environment, such as R or Python, and then execute scripts/notebooks that reside on our computer on data that's on our computer.

Our main point of access to either the HPC cluster is going to be our computer.

5.3.2 Key Players in HPC

Let's contrast our view of local computing with the key players in the HPC cluster (Figure 5.2).

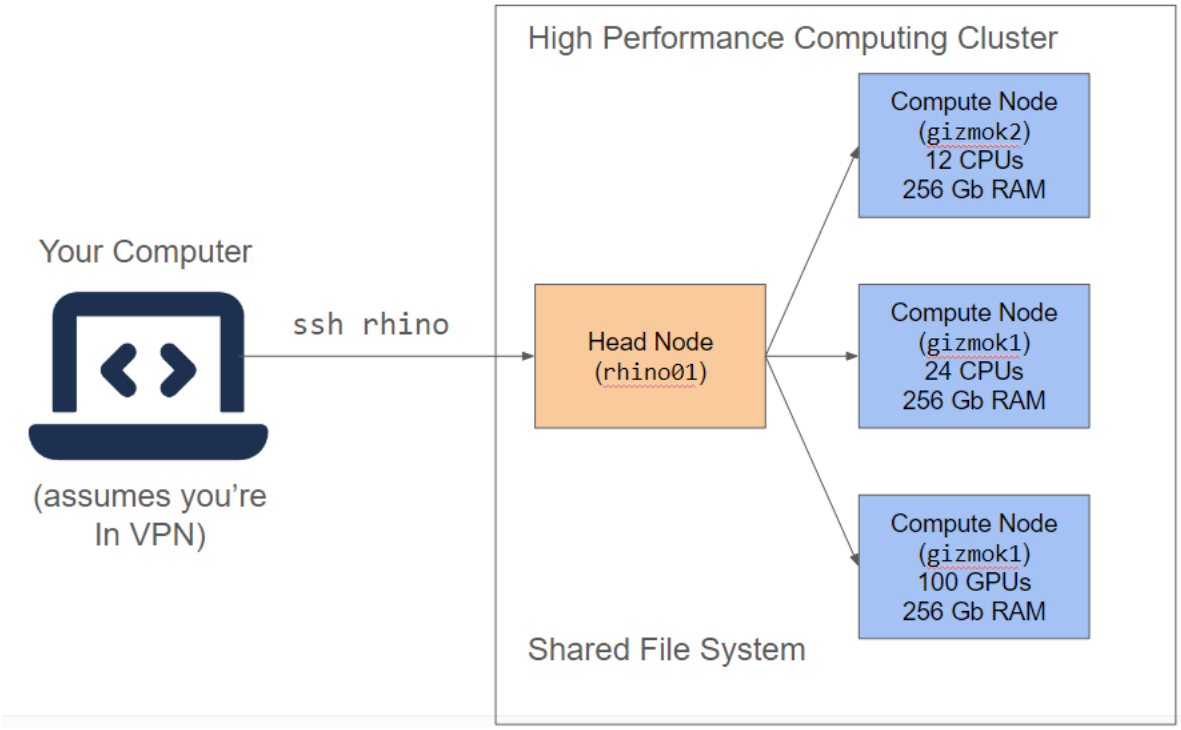


Figure 5.2: Key Players in HPC

- **Our Machine** - We interact with the HPC via the terminal installed on our machine. When we utilize HPC resources, we request them from our own computer using commands from the dx toolkit.
- **Head Node** - The "boss" of the cluster. It keeps tract of which worker nodes is doing what, and which nodes are available for allocations. Our request gets sent to the HPC cluster, and given availability, it will grant access to temporary worker.
- **Worker Node** - A temporary machine that comes from a pool of available machines in the cluster. We'll see that it starts out as a blank slate, and we need to establish a *software environment* to run things on a worker.

- **Shared Filesystem** A distributed filesystem that can be seen by all of the nodes in the cluster. Our scripts and data live here.

5.3.3 Further Reading

- Working on a remote HPC system is also a good overview of the different parts of HPC.

> **For Fred Hutch Users**
>
> The `gizmo` cluster at Fred Hutch actually has 3 head nodes called `rhino` (`rhino01`, `rhino02`, `rhino03`) that are high spec machines (70+ cores, lots of memory). You can run jobs on these nodes, but be aware that others may be running jobs here as well.
> The worker nodes on the `gizmo` cluster all have names like `gizmoj6`, depending on their architecture. You can request certain kinds of nodes in an allocation in several ways:
>
> 1. When you use `grabnode` (Section 14.2.1)
> 2. In your request when you run `srun` or `sbatch`
> 3. In your WDL or Nextflow Workflow.

> **FH Users: Launching Jobs on `gizmo`**
>
> In a SLURM cluster like `gizmo`, SLURM (Simple Linux Utility for Resource Management) is software that runs on the head node that manages jobs on individual nodes.
> The two main mechanisms for running SLURM jobs are `srun` (used for single jobs) and `sbatch` (used for multiple related jobs, such as aligning a set of FASTA files).
> When scaling to a larger number of files, we do recommend that you use a workflow manager such as Cromwell, or PROOF, or Nextflow for batching files.

5.4 Sequence of Events of Running a Job

Let's run through the order of operations of running a job on the HPC cluster. Let's focus on running an aligner (BWA-MEM) on a FASTQ file. Our output will be a .BAM (aligned reads) file.

Let's go over the order of operations needed to execute our job on the HPC cluster (Figure 5.3).

A. **Start a job using `srun` to send a request to the cluster.** In order to start a job, we will need two things: software (`samtools`), and a file to process from the shared filesystem (not shown). When we use `srun`, a request is sent to the cluster.

B. **Head node requests for a worker from available workers; worker made available on cluster.** In this step, the head node looks for a set of workers that can meet our needs.

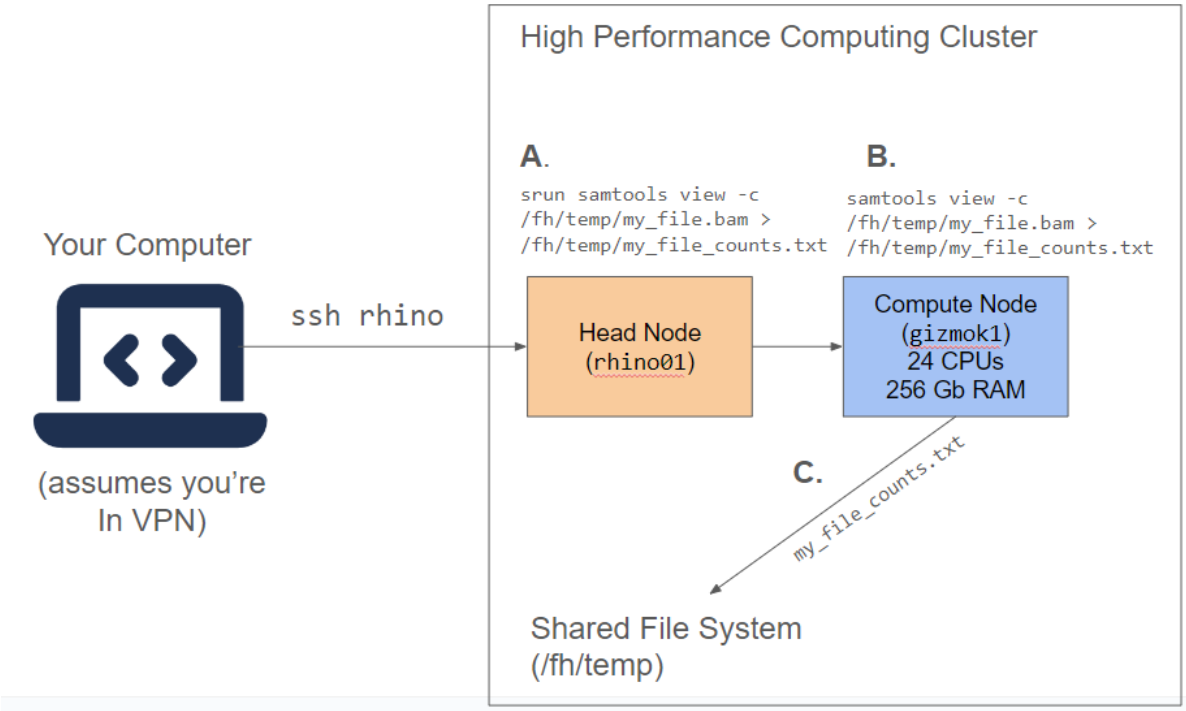


Figure 5.3: Order of Operations

Then the computations run on the worker; output files are generated.** Once our app is ready and our file is transferred, we can run the computation on the worker.

C. **Output files transferred back to project storage.** Any files that we generate during our computation (`53525342.bam`) must be transferred back into the shared filesystem.

When you are working with an HPC cluster, especially with batch jobs, keep in mind this order of execution. Being familiar with how the key players interact on the cluster is key to running efficient jobs.

5.4.1 Key Differences with local computing

As you might have surmised, running a job on the HPC cluster is very different from computing on your local computer. Here are a few key differences:

1. We don't own the worker machine, we only have temporary access to it. A lot of the complications of running HPC computations comes from this.
2. We have to be explicit about what kind of machine we want. We'll talk much more about this in terms of machine types and classifieds.

5.5 The Shared Filesystem

Clusters often have a shared filesystem to make things easier. These filesystems can be accessed by all the nodes in a cluster and are designed for fast file transfers and reading. One example of a filesystem is Lustre.

Think about that: it's like having an SSD attached to all of the nodes. But how does the shared storage work?

The filesystem is distributed such that each set of nodes has a relatively fast access to the files on the system. The data itself is *sharded*, or broken up, and distributed among the storage servers that provide access to the files.

> **For FH Users**
>
> There are three main filesystems you will probably use:
>
> - `/home/` - usually where your scripts will live
> - `/fh/fast/` - Where data lives. You will usually transfer data files over to `/hpc/temp/` and when you generate results, transfer them from `/hpc/temp/` back to `/fh/fast/`
> - `/hpc/temp/` - A temporary filesystem. Don't store files here long term - mostly use this as a faster system to do computations on.

5.5.1 Further Reading

- Transferring Files is a nice overview of the ways to transfer files to and from a remote system.

5.6 Requesting Machines

How do you request a set of machines on the HPC? There are multiple ways to do so:

- Open an Interactive shell on a Node
- As part of a job using `srun` or `sbatch`

In general, we recommend trying to use SLURM as much as possible with `srun` or `sbatch`. The larger recommendation overall is that we encourage you to use a workflow runner such as PROOF or Nextflow.

5.6.1 Scattering: Distribute the Work

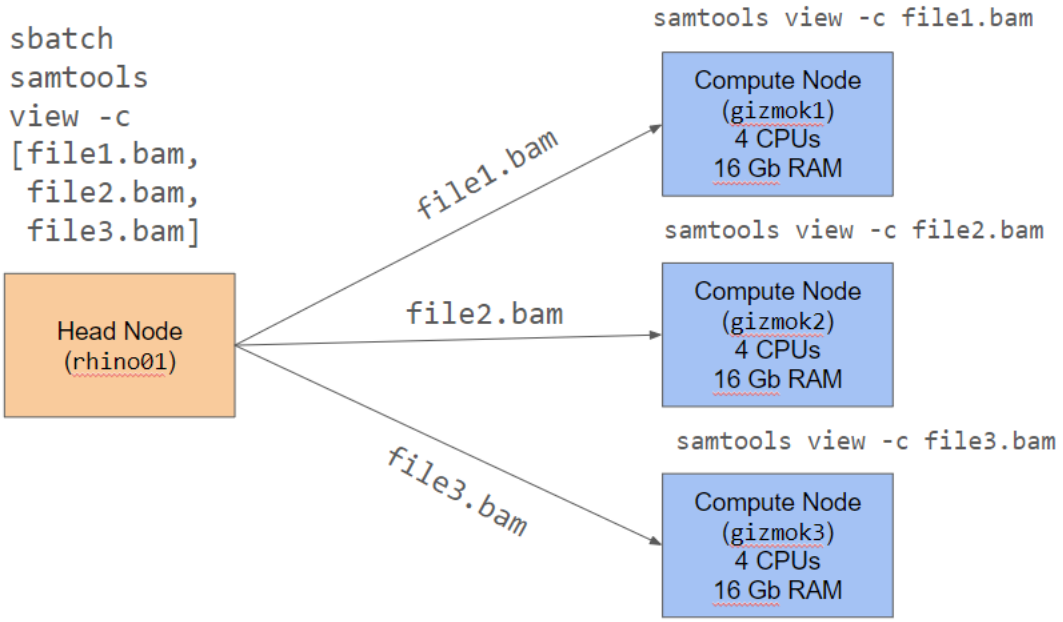


Figure 5.4: The scattering process

So far, everything we've seen so far can be run on a single computer. In the cluster, we have access to higher spec'ed machines, but using the cluster in this way doesn't take advantage of the efficiency of *distributed computing*, or dividing the work up among multiple worker nodes.

We can see an example of this in Figure 5.4. In distributed computing, we break our job up into smaller parts. One of the easiest way to do this is to split up a list of files (`file1.bam`, `file2.bam`, `file3.bam`) that we need to process, process each file separately on a different node, and then bring the results back together. Each node is only doing part of the work, but because we have multiple nodes, it is getting done 3 times faster.

You can orchestrate this process yourself with tools such as `sbatch`, but it is usually much easier to utilize workflow runners such as Cromwell/PROOF (for `.wdl` files) or Nextflow (for `.nf` files), because they automatically handle saving the results of intermediate steps.

Trust me, it is a little more of a learning curve to learn Cromwell or Nextflow, but once you know more about it, the automatic file management and node management makes it much easier in the long run.

Nodes versus CPUs

One thing that confused me was understanding the difference between requesting a system with multiple cores versus requesting multiple nodes.
Cores roughly correspond to processors, so a 24-core allocation is a single node that has 24 CPUs.
Nodes correspond to machines - so a 24 node allocation is 24 machines.
The reason why this is important is that you use them differently - we use scatter to utilize a 24 node allocation, whereas we can use multicore packages such as `{parallel}` and `mcapply()` to utilize a multi-core system.
In general, scatter over multiple nodes is handled by `sbatch` or your workflow runner.

For FH Users: Running Workflows

At Fred Hutch, we have two main ways to run workflows on `gizmo`: Cromwell/MiniWDL and NextFlow. Cromwell users have a nifty GUI to run their workflows called PROOF.

5.7 What's Next?

We'll leverage this knowledge to actually start batching on the `gizmo` cluster.

6 Week 2: Batch Processing and Submitting Jobs

6.1 Exercises

Open up the exercises here.

6.2 Learning Objectives

- **Execute** a script to run over a list of files on one system
- **Utilize** globs to specify multiple files in your script
- **Batch Process** files to run locally and on the HPC cluster

6.3 Using `for` loops to cycle through files

A very common pattern is cycling through multiple files in a folder and applying the same script or command to them.

There is a simple method for batch processing a bunch of files: a `for` loop. In our case, a for loop takes a list of file paths (such as a list of FASTA files we want to process), and performs the same task for each element of the list.

```
for file in 01_assignment.qmd 02_scripting.qmd    ①
do                                                 ②
  wc $file                                         ③
done                                               ④
```

① Cycle through the list of `01_assignment.qmd` and `02_scripting.qmd`.
② Start of instructions
③ Count the words in each `.qmd` file using `wc`
④ End of instructions

The `do` and `done` sandwich the instructions we want to do to each file in our list. `wc` will produce a word count of these two files.

If we run this in the `bash_for_bio/` folder, we'll get the following:

```
     26      96     656 01_assignment.qmd
    493    2609   16206 02_scripting.qmd
```

6.4 globs: selecting multiple files

However, typing each element of our list is going to be difficult - can we select files in a different way?

We can use `globs`, or wildcard characters to select multiple files with a criteria.

For example, `*.qmd` will list all of the `.qmd` files in the `bash_for_bio` directory.

```
#| filename: ./scripts/week3/batch_on_rhino.sh
#!/bin/bash
for file in ./data/*.fastq                                    ①
do                                                            ②
  wc $file                                                    ③
done                                                          ④
```

① Start `for` loop and cycle through all `.qmd` files
② Start of instructions
③ Count the words in each `.qmd` file using `wc`
④ End of instructions

If we run this in our repository, we get something similar to this:

```
3220   3220 142485 ./data/CALU1_combined_final.fastq
2484   2484 109917 ./data/HCC4006_final.fastq
1836   1836 81243 ./data/MOLM13_combined_final.fastq
```

The `*.fasta` (the wildcard operator, also known as a) can be used in various ways. For example, if our files are in a folder called `data/`, we could specify:

```
for file in ./data/*.fastq
do
  wc $file
done
```

6.4.1 Try it out

`./scripts/week3/batch_on_rhino.sh`

A common pattern: a folder with only one type of file in them

One thing that makes it easier to process a bunch of files is to have the data be in the same folder, with nothing else in them.
For example, I might create a `fastq/` folder where I store my data, so I can pass the glob `fastq/*` to process the files in that.

```
#!/bin/bash
module load BWA
module load SAMtools
FASTA_LOCATION=""
OUTPUT_FOLDER="/home/tladera2/project_x/bam_files/"
for file in fastq/*
do
  bwa mem ${FASTA_LOCATION} file > ${OUTPUT_FOLDER}${file}.bam
done
module purge
```

6.4.2 For more info on globs

See page 12 in Bite Size Bash.

Selecting files with complicated patterns: Regular Expressions

At some point, globs are going to be inadequate depending on how you store files. At that point, you will probably need to learn regular expressions, which is a much more powerful way of describing search patterns.
I will be honest and say this is one thing LLMs are very good at. But you should have the vocabulary to prompt them, including:

1. Literal Characters
2. Metacharacters (including how to escape special characters)
3. Character classes (specifying groups of characters to match)
4. Capture Groups, including named capture groups
5. Quantifiers
6. Logical expressions
7. Anchors (matching positions)

Knowing these general concepts will help you with your LLM prompts. And always test

> your regular expressions to make sure that they are capturing the file patterns you expect.

6.5 Batching on HPC

Now we can start to do more advanced things on the HPC cluster: use one machine to process each file. We will use a slightly different mechanism to cycle through files: the .

Let's start out with scripts.

6.5.1 SLURM Scripts

SLURM scripts are a special kind of shell script that contain additional information for the SLURM manager. This includes:

1. Number of nodes (machines) to request
2. Memory and CPU requirements for each machine

We specify these using a special kind of comment: SLURM directives. Directives begin a line with `#SBATCH`:

```
#SBATCH --nodes=1
```

In this example, we are specifying the number of nodes.

Note that because directives begin with a `#` - they are treated like comments by `bash`, but are usable by `SLURM`.n

6.5.2 SLURM Directives

We are able to set some configuration on running our jobs.

```bash
#!/bin/bash
#SBATCH --nodes=3                                                    ①
#SBATCH --array=1-3                                                  ②
sleep 60
echo "${SLURM_ARRAY_TASK_ID} job"                                    ③
```

① request 1 node
② start an array

More about directives

Much more information about the kinds of directives that you can specify in a SLURM script is available here: https://www.osc.edu/supercomputing/batch-processing-at-osc/slurm_directives_summary
The most important directive you should be aware of is how many nodes you need to request, and how much memory your jobs will need. `gizmo` does not use the memory parameter. Instead, use the number of cores as a proxy for memory. Much more information is here https://sciwiki.fredhutch.org/scicomputing/compute_jobs/#memory

All About Jobs

There is a ton of information that I'm not including about the `gizmo` cluster. For much more info on submitting jobs, please refer to https://sciwiki.fredhutch.org/scicomputing/compute_jobs/.

6.5.3 Job Arrays

This line:

```
#SBATCH --array=1-3
```

Will create a job array. This will create a variable called `$SLURM_ARRAY_TASK_ID` that will cycle through the numbers 1-3. Each Task ID corresponds to a different subjob. Let's try a simpler script to show what's going on:

```
#!/bin/bash
#SBATCH --array=1-3
#SBATCH --nodes=1
echo "${SLURM_ARRAY_TASK_ID} job"
```

This is a minimal script that will execute 3 subjobs. It will cycle through the job array and print the array number for each job.

```
sbatch sbatch_test.sh
```

On submitting, we will get a message like this (your job number will be different):

```
Submitted batch job 26328834
```

This will run very quickly on the three nodes. And if we look for the output files:

```
ls -l slurm-26328834*
```

We will get the following output:

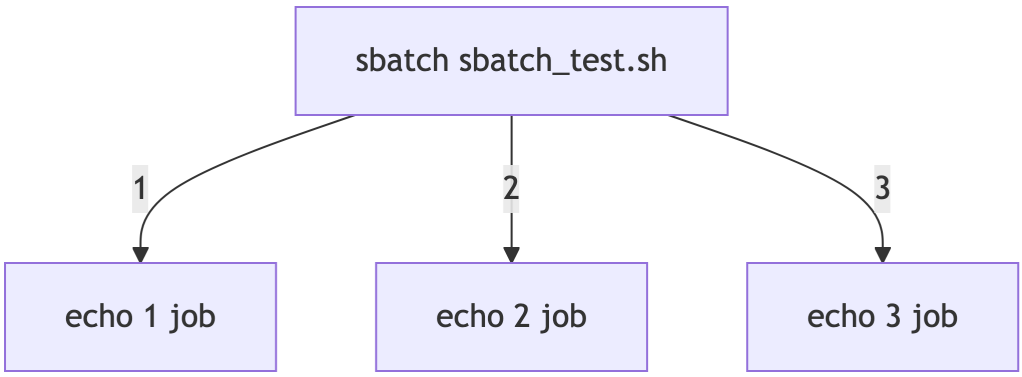
```
-rw-rw---- 1 tladera2 g_tladera2 8 Jul 15 13:50 slurm-26328834_1.out
-rw-rw---- 1 tladera2 g_tladera2 8 Jul 15 13:50 slurm-26328834_2.out
-rw-rw---- 1 tladera2 g_tladera2 8 Jul 15 13:50 slurm-26328834_3.out
```

Taking a look at one of these files using `cat`:

```
cat slurm-26328834_3.out
```

We'll see this:

```
3 job
```



What happened here? `sbatch` submitted our job array as 3 different subjobs to 3 different nodes under a single job id. Each node then outputs a file with the subjob id that contains the job number.

6.5.4 Try it out

Try running

```
sbatch ./scripts/week3/sbatch_test.sh
```

And look at the resulting `.out` files.

6.5.5 Processing lists of files using Job Arrays

So now we know that `${SLURM_ARRAY_TASK_ID}` will let us specify a subjob within our script, how do we use it in our script?

Say, we have a list of 3 files in our directory `data/`, and we can list them by using `../../data/*.fastq`. We can use the `${SLURM_ARRAY_TASK_ID}` as an index to specify a different file.

The one caveat is that we need to know the number of files beforehand.

This script will run our `run_bwa.sh` on 3 separate files on 3 separate nodes:

```
#!/bin/bash
#SBATCH --nodes=3                                    ①
#SBATCH --array=1-3                                  ②
#SBATCH --mem-per-cpu=1gb                            ③
#SBATCH --time=00:10:00                              ④
file_array=(../../data/*.fastq)                      ⑤
ind=$((SLURM_ARRAY_TASK_ID-1))                       ⑥
current_file=${file_array[$ind]}                     ⑦
./run_bwa.sh $current_file
```

① Initialize job array to range from 1 to 10
② list all files in `../..data/` with extension `.fastq`. Assign it to the array `$file_array`.
③ For current task id, calculate the appropriate index (we have to subtract 1 because bash arrays begin at `0`)
④ Pull the current file path given the task id, assign it to `$current_file`.
⑤ Run `run_bwa.sh` on `$current_file`.

We run this on `rhino` using this command (we are in `~/bash_for_bio/scripts/week3/`)

```
sbatch sbatch_bwa.sh
```

We'll get the response:

```
Submitted batch job 35300989
```

If we take a look at the job queue using `squeue`, we'll get something like this:

```
squeue -u tladera2
```

```
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
        35300989_1 campus-ne sbatch_b tladera2  R       0:10      1 gizmok34
        35300989_2 campus-ne sbatch_b tladera2  R       0:10      1 gizmok40
        35300989_3 campus-ne sbatch_b tladera2  R       0:10      1 gizmok79
```

We can see our three subjobs, which are indicated by the Job IDs `35300989_1`, `35300989_2`, and `35300989_3`.

```
ladera2@rhino02:~/bash_for_bio/scripts/week3$ ls -l
```

```
total 312
-rw-rw---- 1 tladera2 g_tladera2 211895 Sep 26 10:29 CALU1_combined_final.sam
-rw-rw---- 1 tladera2 g_tladera2 160489 Sep 26 10:29 HCC4006_final.sam
-rw-rw---- 1 tladera2 g_tladera2 121509 Sep 26 10:29 MOLM13_combined_final.sam
-rwxrwxrwx 1 tladera2 g_tladera2    590 Sep 26 09:45 run_bwa.sh
-rwxrwxrwx 1 tladera2 g_tladera2    256 Sep 26 10:28 run_sbatch.sh
-rw-rw---- 1 tladera2 g_tladera2    614 Sep 26 10:29 slurm-35300992_1.out
-rw-rw---- 1 tladera2 g_tladera2    579 Sep 26 10:29 slurm-35300992_2.out
-rw-rw---- 1 tladera2 g_tladera2    619 Sep 26 10:29 slurm-35300992_3.out
```

And you'll see we generated our SAM files for each sample! Neat. There are also the `.out` files from each subjob as well, which will contain the for each subjob.

You can see that we outputted our files to the `scripts/week3/` directory. It's part of your job in the exercises to adapt `run_sbatch.sh` and `run_bwa.sh` to output to a directory of your choosing.

> **Why isn't my job launching?**
>
> The `gizmo` cluster is used by a lot of people at the Hutch, so it gets busy from time to time.
> Don't worry, your jobs will be eventually processed if your requests are reasonable.
> If you want to look at the jobs that a particular user is running, you can use the -u flag.
> Try squeue -u on one of the users in the queue, for example `tladera2`:
>
> ```
> squeue -u tladera2
> ```
>
> You will usually use `squeue -u` on your own username, so you can see the status of your jobs. This is an example from my previous career at OHSU (I didn't have time to generate one on `gizmo`)
>
> ```
> JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
> ```

```
           3970834 very_long KIRC.bla    wooma  R 11-
04:21:09      1 exanode-7-15
           3970835 very_long OV.blast    wooma  R 11-
04:20:32      1 exanode-7-5
```

Let's take a look at the output. You can see some pretty useful info: the JOBID, what PARTITION the job is running under, and the STatus.
STatus is really important, because it can tell you whether your job is:
`R` (Running), `PD` (Pending), `ST` (Stopped), `S` (Suspended), `CD` (Completed).
`R` or `PD` status is what you want to see, because that means it's in the queue and will be executed.
For much more info, please check out https://sciwiki.fredhutch.org/scicomputing/compute_jobs/#job-priority and https://sciwiki.fredhutch.org/scicomputing/compute_jobs/#why-isnt-my-job-running.

6.5.6 `scancel`ing a jobarray

As we noted, one of the strengths in using a job array to process multiple files is that they are spawned as *sub* or *child* jobs of a parent job id.

What if we made a mistake? We can use the `scancel` command to cancel the entire set of jobs by giving it our parent job id:

```
scancel 26328834
```

This will cancel all sub jobs related to the parent job. No fuss, no muss.

6.6 What's Next?

Next week we will talk about using Containers on the cluster using `Apptainer`.

7 Week 2 Exercises

7.1 In-class

1. Try it out:

```
./scripts/week3/batch_on_rhino.sh
```

2. Try it out:

```
#| eval: false
sbatch sbatch_test.sh
```

and examine the following output files:

3. Try it out:

```
```

7.2 Homework exercises

Exercises 1, 2, 3 are required for the badge.

1. In the `data/` folder, there is a list of `.csv` files. Modify the code below to process each file.

```
#!/bin/bash
for file in ------
do
  head ------
done
```

2. Modify the code in 1) to write a `for` loop in a script to count the number of lines for each `.csv` file. The output should be:

```
echo "${file}\t(wc -l $file)\n"
```

Run the script and redirect your output to `files_out.txt`. Put your bash command you used below:

3. Modify the below script to process the `.sam` files in `./data/`. Save it as `sbatch_sam.sh` and run it. How many files were generated?

```
#!/bin/bash
#SBATCH --array=1-________
module load SAMtools
file_array=(./data/*.sam)
ind=$((SLURM_ARRAY_TASK_ID-1))
current_file=${file_array[$ind]}
samtools view -c $current_file > ${current_file}.counts.out
```

8 Week 3 Reading: Containers

8.1 Containers

We already learned about software modules (Section 3.5) on the `gizmo` cluster. There is an alternative way to use software: pulling and running a software .

8.1.1 What is a Container?

A container is a self-contained unit of software. It contains everything needed to run the software on a variety of machines. If you have the container software installed on your machine, it doesn't matter whether it is MacOS, Linux, or Windows - the container will behave consistently across different operating systems and architectures.

The container has the following contents:

- **Software** - The software we want to run in a container. For bioinformatics work, this is usually something like an aligner like `bwa`, or utilities such as `samtools`
- **Software Dependencies** - various software packages needed to run the software. For example, if we wanted to run `tidyverse` in a container, we need to have `R` installed in the container as well.
- **Filesystem** - containers have their own isolated filesystem that can be connected to the "outside world" - everything outside of the container. We'll learn more about customizing these with bind paths (Section 9.3.3).

In short, the container has everything needed to run the software. It is not a full operating system, but a smaller mini-version that cuts out a lot of cruft.

Containers are . They leverage the the file system of their host to manage files. These are called both *Volumes* (the Docker term) and *Bind Paths* (the apptainer term).

8.1.2 Docker vs. Apptainer

There are two basic ways to run Docker containers:

1. Using the Docker software
2. Using the Apptainer software (for HPC systems)

In general, Docker is used on systems where you have a high level of access to the system. This is because `docker` uses a special user group called `docker` that has essentially root level privileges. This is not something to be taken lightly.

This is not the case for HPC systems, which are shared and granting this level of access to many people is not practical. This is when we use (which used to be called Singularity), which requires a much lower level of user privileges to execute tasks. For more info, see Section 9.3 .

Be Secure

Before we get started, security is always a concern when running containers. The `docker` group has elevated status on a system, so we need to be careful that when we're running them, these containers aren't introducing any system vulnerabilities. Note that on HPC systems, the main mechanism for running containers is `apptainer`, which is designed to be more secure.
These are mostly important when running containers that are web-servers or part of a web stack, but it is also important to think about when running jobs on HPC.
Here are some guidelines to think about when you are working with a container.

- **Use vendor-specific Docker Images when possible**.
- **Use container scanners to spot potential vulnerabilities**. DockerHub has a vulnerability scanner that scans your Docker images for potential vulnerabilities. For example, the WILDS Docker library employs a vulnerability scanner and the containers are regularly patched to prevent vulnerabilities.
- **Avoid kitchen-sink images**. One issue is when an image is built on top of many other images. It makes it really difficult to plug vulnerabilities. When in doubt, use images from trusted people and organizations. At the very least, look at the Dockerfile to see that suspicious software isn't being installed.

9 Week 3: Containers

9.1 Containers

We already learned about software modules (Section 3.5) on the `gizmo` cluster. There is an alternative way to use software: pulling and running a software .

9.1.1 What is a Container?

A container is a self-contained unit of software. It contains everything needed to run the software on a variety of machines. If you have the container software installed on your machine, it doesn't matter whether it is MacOS, Linux, or Windows - the container will behave consistently across different operating systems and architectures.

The container has the following contents:

- **Software** - The software we want to run in a container. For bioinformatics work, this is usually something like an aligner like `bwa`, or utilities such as `samtools`
- **Software Dependencies** - various software packages needed to run the software. For example, if we wanted to run `tidyverse` in a container, we need to have `R` installed in the container as well.
- **Filesystem** - containers have their own isolated filesystem that can be connected to the "outside world" - everything outside of the container. We'll learn more about customizing these with bind paths (Section 9.3.3).

In short, the container has everything needed to run the software. It is not a full operating system, but a smaller mini-version that cuts out a lot of cruft.

Containers are . They leverage the the file system of their host to manage files. These are called both *Volumes* (the Docker term) and *Bind Paths* (the apptainer term).

9.1.2 Docker vs. Apptainer

There are two basic ways to run Docker containers:

1. Using the Docker software
2. Using the Apptainer software (for HPC systems)

In general, Docker is used on systems where you have a high level of access to the system. This is because `docker` uses a special user group called `docker` that has essentially root level privileges. This is not something to be taken lightly.

This is not the case for HPC systems, which are shared and granting this level of access to many people is not practical. This is when we use (which used to be called Singularity), which requires a much lower level of user privileges to execute tasks. For more info, see Section 9.3 .

> ⚠ Be Secure
>
> Before we get started, security is always a concern when running containers. The `docker` group has elevated status on a system, so we need to be careful that when we're running them, these containers aren't introducing any system vulnerabilities. Note that on HPC systems, the main mechanism for running containers is `apptainer`, which is designed to be more secure.
> These are mostly important when running containers that are web-servers or part of a web stack, but it is also important to think about when running jobs on HPC.
> Here are some guidelines to think about when you are working with a container.
>
> - **Use vendor-specific Docker Images when possible**.
> - **Use container scanners to spot potential vulnerabilities**. DockerHub has a vulnerability scanner that scans your Docker images for potential vulnerabilities. For example, the WILDS Docker library employs a vulnerability scanner and the containers are regularly patched to prevent vulnerabilities.
> - **Avoid kitchen-sink images**. One issue is when an image is built on top of many other images. It makes it really difficult to plug vulnerabilities. When in doubt, use images from trusted people and organizations. At the very least, look at the Dockerfile to see that suspicious software isn't being installed.

9.2 Pulling a Docker Container and Running It

Let's pull a docker container from the Docker registry. Note we have to specify `docker://` when we pull the container, because Apptainer has its own internal format called SIF.

```
module load Apptainer/1.1.6
apptainer pull docker://biocontainers/samtools:v1.9-4-deb_cv1
apptainer exec \
    --bind /path/to/data:/data \ # <1>
    docker://biocontainers/samtools:v1.9-4-deb_cv1 \
    samtools view -c /mydata/$1 > /mydata/$1.counts.txt
```

9.2.1 Using a Container

In Section 9.1, we learned a little bit about using Apptainer to run a Docker container. Let's try to pull a common container, the Genome Analysis Toolkit (GATK) and run things inside the container.

The first thing we need to do is load Apptainer:

```
module load Apptainer/1.1.6
```

Then we can pull the docker container:

```
apptainer pull docker://biocontainers/samtools:v1.9-4-deb_cv1
```

We can check if we have pulled the docker image by using

```
apptainer cache list
```

Okay, now we have confirmed that we downloaded the apptainer image. Now we can try to execute things with it.

```
apptainer exec \
    --bind /path/to/data:/data \ # <1>
    docker://biocontainers/samtools:v1.9-4-deb_cv1 \ # <2>
    samtools view -c /mydata/$1 > /mydata/$1.counts.txt                    ③
```

① Bind path (see Section 9.3.3)
② Docker image we have downloaded
③ `samtools` command to run.

It's worth trying this once to make sure you understand how all of the pieces are connected. In general, I do recommend using a workflow runner (Section 12.3) instead, because it helps manage all of these details, and it makes reading files easier.

9.2.2 Bind Paths

One thing to keep in mind is that containers have their own filesystem. They can only read and write to folders in the external filesystem that you give them access to with *bind paths*. The one exception is the current working directory.

For more info about bind paths see Section 9.3.3.

9.3 Testing code in a container

I think the hardest thing about working with containers is wrapping your head around the indirectness of them. You are running software with its own internal filesystem and the challenges are getting the container to read files in folders/paths outside of its own filesytem, as well as outputting files into those outside folders.

The best way to understand containers is to open a shell in a container. Remember, containers are self-contained mini operating systems, and the most important thing to understand what they isolate from the rest of the system, and how to get files into and out of the container.

In this section, we talk about testing scripts in a container using `apptainer`. We use `apptainer` (formerly Singularity) in order to run Docker containers on a shared HPC system. This is because Docker itself requires root-level privileges, which is not secure on shared systems.

In order to do our testing, we'll first pull the Docker container, map our bind point (so our container can access files outside of its file system), and then run scripts in the container.

Even if you aren't going to frequently use Apptainer in your work, I recommend trying an interactive shell in a container at least once or twice to learn about the container filesystem and conceptually understand how you connect it to the external filesystem.

9.3.1 Pulling a Docker Container

Let's pull a docker container from the Docker registry. Note we have to specify `docker://` when we pull the container, because Apptainer has its own internal format called SIF.

```
module load Apptainer/1.1.6
apptainer pull docker://biocontainers/samtools:v1.9-4-deb_cv1
```

9.3.2 Opening a Shell in a Container with `apptainer shell`

When you're getting started, opening a shell using Apptainer can help you test out things like filepaths and how they're accessed in the container. It's hard to get an intuition for how file I/O works with containers until you can see the limited view from the container.

By default, apptainers can see your current directory and navigate to the files in it.

You can open an Apptainer shell in a container using `apptainer shell`. Remember to use `docker://` before the container name. For example:

```
module load Apptainer/1.1.6
apptainer shell docker://biocontainers/samtools:v1.9-4-deb_cv1
```

This will load the `apptainer` module, and then open a Bash shell in the container using `apptainer shell`. Once you're in the container, you can test code, especially seeing whether your files can be seen by the container (see Section 9.3.3). 90% of the issues with using Docker containers has to do with bind paths, so we'll talk about that next.

Once you're in the shell, you can take a look at where `samtools` is installed:

```
which samtools
```

Note that the container filesystem is isolated, and we need to explicitly build connections to it (called bind paths) to get files in and out. We'll talk more about this in the next section.

Once we're done testing scripts in our containers, we can exit the shell and get back into the node.

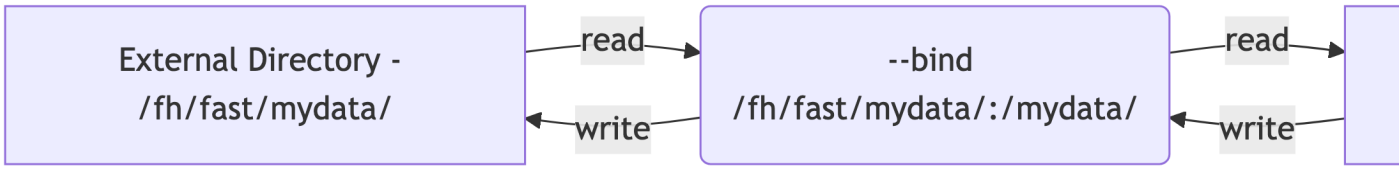
```
exit
```

9.3.3 Using bind paths in containers

One thing to keep in mind is that every container has its own filesystem. One of the hardest things to wrap your head around for containers is how their filesystems work, and how to access files that are outside of the container filesystem. We'll call any filesystems outside of the container *external filesystems* to make the discussion a little easier.

By default, the containers have access to your current working directory. We could make this where our scripts live (such as `/home/tladera2/`), but because our data is elsewhere, we'll need to specify that location (`/fh/fast/mylab/`) as well.

The main mechanism we have in Apptainer to access the external filesystem are *bind paths*. Much like mounting a drive, we can bind directories from the external filesystem using these bind paths.



I think of bind paths as "tunnels" that give access to particular folders in the external filesystem. Once the tunnel is open, we can access data files, process them, and save them using the bind path.

Say my data lives in `/fh/fast/mydata/`. Then I can specify a bind point called `mydata/` in my `apptainer shell` and `apptainer run` commands so my container can access the files in that directory. Then in the container, we can access the files through the `mydata/` bind path.

We can do this with the `--bind` option:

```
apptainer shell --bind /fh/fast/mydata:/mydata docker://biocontainers/samtools:v1.9-4-deb_cv
```

Note that the bind syntax doesn't have the trailing slash (`/`). That is, note that it is:

```
--bind /fh/fast/mydata: ....
```

Rather than

```
--bind /fh/fast/mydata/: ....
```

Now our `/fh/fast/mydata/` folder will be available as `/mydata/` in my container. We can read and write files to this bind point. For example while in the container's shell, I'd refer to the `.bam` file `/fh/fast/mydata/my_bam_file.bam` as:

```
samtools view -c /mydata/my_bam_file.bam
```

> **Opening a Shell in a Docker Container with Docker**
>
> For the most part, due to security reasons, we don't use `docker` on HPC systems. In short, the `docker` group essentially has root-level access to the machine, and it's not a good for security on a shared resource like an HPC.
> However, if you have admin level access (for example, on your own laptop), you can open up an interactive shell with `docker run -it`:
>
> ```
> docker run -it biocontainers/samtools:v1.9-4-deb_cv1 /bin/bash
> ```
>
> This will open a bash shell much like `apptainer shell`. Note that volumes (the docker equivalent of bind paths) are specified differently in Docker compared to Apptainer.

> **WDL makes this way easier**
>
> A major point of failure with Apptainer scripting is when our scripts aren't using the right bind paths. It becomes even more complicated when you are running multiple steps. This is one reason we recommend writing WDL Workflows and a (such as or Sprocket) to run your workflows. You don't have to worry that your bind points are setup correctly, because they are handled by the workflow manager.

9.3.4 Executing in the Apptainer Shell

Ok, now we have a bind point, so now we can test our script in the shell. For example, we can see if we are invoking `samtools` in the correct way and that our bind points work.

```
samtools view -c /mydata/my_bam_file.bam > /mydata/bam_counts.txt
```

Again, trying out scripts in the container is the best way to understand what the container can and can't see.

9.3.5 Exiting the container when you're done

You can `exit`, like any shell you open. You should be out of the container. Confirm by using `hostname` to make sure you're out of the container.

9.3.6 Testing outside of the container

Let's take everything that we learned and put it in a script that we can run on the HPC:

```
# Script to samtools view -c an input file:
# Usage: ./run_sam.sh <my_bam_file.bam>
# Outputs a count file: my_bam_file.bam.counts.txt
#!/bin/bash
module load Apptainer/1.1.6
apptainer run --bind /fh/fast/mydata:/mydata \
    docker://biocontainers/samtools:v1.9-4-deb_cv1 \
    samtools view -c /mydata/$1 > /mydata/$1.counts.txt
#apptainer cache clean
module purge
```

We can use this script by the following command:

```
./run_sam.sh chr1.bam
```

And it will output a file called `chr1.bam.counts.txt`.

> **Apptainer Cache**
>
> The apptainer cache is where your docker images live. They are translated to the native apptainer `.sif` format.

> You can see what's in your cache by using
>
> `apptainer cache list`
>
> By default the cache lives at `~/.apptainer/cache`.
> If you need to clear out the cache, you can run
>
> `apptainer cache clean`
>
> to clear out the cache.
> There are a number of environment variables (Section 15.3) that can be set, including login tokens for pulling from a private registry. More information is here.

9.3.7 The WILDS Docker Library

The Data Science Lab has a set of Docker containers for common Bioinformatics tasks available in the WILDS Docker Library. These include:

- `bwa mem`
- `samtools`
- `gatk`
- `bcftools`
- `manta`
- `cnvkit`
- `deseq2`

Among many others. Be sure to check it out before you start building your own containers.

9.4 What's Next?

Next week we will discuss using `MiniWDL` (a workflow manager) to process files through multi-step workflows in and run WDL files using PROOF.

10 Week 3: Assignment

10.1 In-class

1. Pull the `samtools` Docker container:

```
module load Apptainer/1.1.6
apptainer pull docker://biocontainers/samtools:v1.9-4-deb_cv1
```

2. Open a shell in the Docker container:

```
apptainer shell docker://biocontainers/samtools:v1.9-4-deb_cv1
```

Try accessing files in your `/home/username/` directory:

```
ls -l /home/laderast/bash_for_bio/
```

That didn't work - exit the shell:

```
exit
```

3. Bind your directory so that the Docker container can see it:

```
apptainer shell --bind /user/tladera2/bash_for_bio:/bash_for_bio docker://biocontainers/samto
```

Try accessing files in `bash_for_bio/`

```
samtools view -c bash_for_bio/data/MOLM13_combined_final.sam > bash_for_bio/MOLM13_counts.tx
```

```
exit
```

4. Try running the command using `apptainer exec`:

```
apptainer exec \
    --bind /users/tladera2/bash_for_bio:/bash_for_bio \
    docker://biocontainers/samtools:v1.9-4-deb_cv1 \
    samtools view -c bash_for_bio/data/MOLM13_combined_final.sam > \
    /bash_for_bio/MOLM13_combined_final.counts.txt
```

10.2 Homework

1. Adapt the for loop in this script to use `apptainer exec`. You can use an ubuntu container for this.

```
#!/bin/bash
for file in ./data/*.fastq
do
  wc $file
done
```

2. Modify `run_bwa.sh` in `week3/` to use apptainer for bwa. Hints: you will need to load Apptainer, and use `apptainer exec`. To make things easier, pull the bwa container first.

```
#!/bin/bash
module load BWA/0.7.17-GCCcore-11.2.0
input_fastq=${1}
# strip path and suffix
base_file_name="${input_fastq%.fastq}"
base_file_name=${base_file_name##*/}
echo "running $input_fastq"
sample_name="SM:${base_file_name}"
read_group_id="ID:${base_file_name}"
platform_info="PL:Illumina"
ref_fasta_local="/shared/biodata/reference/iGenomes/Homo_sapiens/UCSC/hg19/Sequence/BWAIndex/

bwa mem \
      -p -v 3 -M \
      -R "@RG\t${read_group_id}\t${sample_name}\t${platform_info}" \
      "${ref_fasta_local}" "${input_fastq}" > \
      "${base_file_name}.sam"

module purge
```

Run the `run_bwa.sh` script on one of the files to ensure that it works.

Try using `week3/run_sbatch.sh` on the files in the `data/` directory. Were there any modifications you needed to make to `run_sbatch.sh`?

11 Week 4 Reading: Pipes/JSON

11.1 Using pipes: STDIN, STDOUT, STDERR

We will need to use pipes to chain our commands together. Specifically, we need to take a command that generates a list of files on the cluster shared filesystem, and then spawns individual jobs to process each file. For this reason, understanding a little bit more about how pipes (`|`) work in Bash is helpful.

If we want to understand how to chain our scripts together into a pipeline, it is helpful to know about the different streams that are available to the utilities.

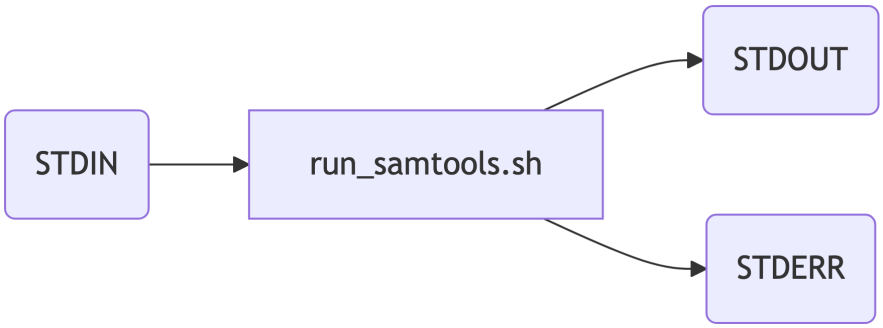


Figure 11.1: Inputs/outputs to a script

Every script has three streams available to it: Standard In (STDIN), Standard Out (STDOUT), and Standard Error (STDERR) (Figure 16.1).

STDIN contains information that is directed to the input of a script (usually text output via STDOUT from another script). This includes arguments we pass onto the script.

Why do these matter? To work in a Unix pipeline, a script must be able to utilize STDIN, and generate STDOUT, and STDERR.

Specifically, in pipelines, STDOUT of a script (here it's `run_bwa.sh`) is directed into STDIN of another command (here `wc`, or word count)

We will mostly use STDOUT in our bash scripts, but STDERR can be really helpful in debugging what's going wrong.

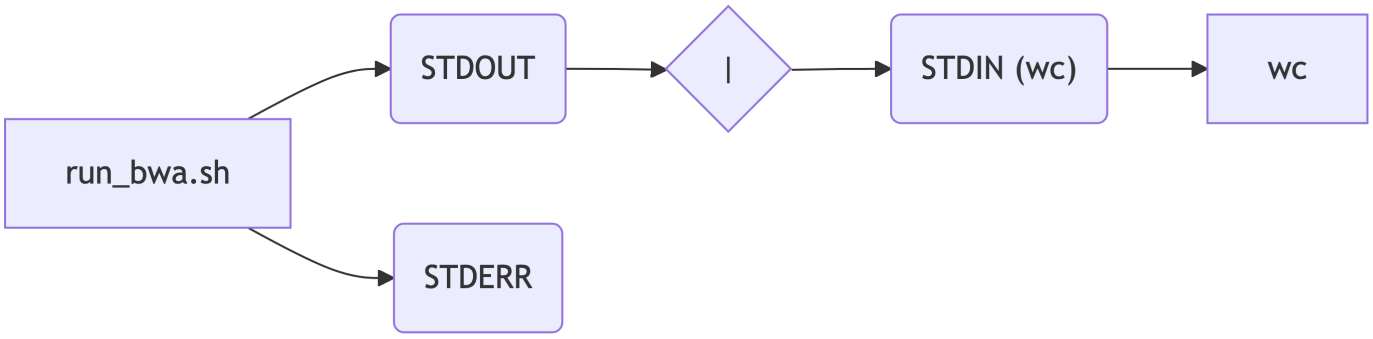


Figure 11.2: Piping a script `run_samtools.sh` into another command (`wc`)

11.1.1 > (redirects)

Sometimes you want to direct the output of a script (STDOUT) to a text file. A lot of bioinformatics tools output to STDOUT, and we need a way to save the results, or pass the results onto another program.

Enter `>`, a redirect. We can put `>` after our command followed by a file path to save the output.

```
samtools view -c my_bam_file.bam > counts.txt
```

Here we are redirecting the output of `samtools view` into the `counts.txt` file. Note that everytime that we run the script, we will overwrite the current contents of the `counts.txt` file. Sometimes that is not what you want.

There is another kind of redirect, `>>`, that will append (that is, add to the end of a file). If the file does not exist, it will be created. But if it does exist, the output will be added to the end of the file. I rarely use this, however.

Much more information about redirects can be found here: https://www.geeksforgeeks.org/linux-unix/input-output-redirection-in-linux/ and in Bite Sized Bash (page 13).

> **Why this is important on the Cluster**
>
> We'll use pipes and pipelines not only in starting a bunch of jobs using batch scripting on our home computer, but also when we are processing files within a job.
> Pipes are at the heart of multi-stage workflows. They allow us to specify multiple steps in processing a file.

11.1.2 For more info about pipes and pipelines

- Bite Size Bash Page 13 for more about redirects
- Bite Size Bash Page 20 for more about pipes
- https://swcarpentry.github.io/shell-novice/04-pipefilter/index.html
- https://datascienceatthecommandline.com/2e/chapter-2-getting-started.html?q=stdin#combining-command-line-tools

11.2 What is JSON?

One requirement for running workflows is basic knowledge of JSON.

JSON is short for **J**ava**S**cript **O**bject **N**otation. It is a format used for storing information on the web and for interacting with (APIs).

11.2.1 How is JSON used?

JSON is used in multiple ways:

- Submitting Jobs with complex parameters/inputs

So having basic knowledge of JSON can be really helpful. JSON is the common language of the internet.

11.2.2 Elements of a JSON file

Here are the main elements of a JSON file:

- **Key:Value Pair**. Example: `"name": "Ted Laderas"`. In this example, our key is "name" and our value is "Ted Laderas"
- **List** `[]` - a collection of values. All values have to be the same data type. Example: `["mom", "dad"]`
- **Object** `{}` - A collection of key/value pairs, enclosed with curly brackets (`{}`).

> **Check Yourself**
>
> What does the `names` value contain in the following JSON? Is it a list, object or key:value pair?
>
> ```
> {
>  "names": ["Ted", "Lisa", "George"]
> }
> ```

> **Answer**
>
> It is a list. We know this because the value contains a `[]`.
>
> ```
> {
>  "names": ["Ted", "Lisa", "George"]
> }
> ```

11.2.3 JSON Input Files

When you are working with WDL, it is easiest to manage files using JSON files. Here's the example we're going to use from the ww-fastq-to-cram workflow.

```
#| eval: false
#| filename: "json_data/example.json"
{
  "PairedFastqsToUnmappedCram.batch_info": [
    {
      "dataset_id": "TESTFASTQ1",
      "sample_name": "HG02635",
      "library_name": "SRR581005",
      "sequencing_center": "1000-Genomes",
      "filepaths": [{
        "flowcell_name": "20121211",
        "fastq_r1_locations": ["tests/data/SRR581005_1.ds.fastq.gz"],
        "fastq_r2_locations": ["tests/data/SRR581005_2.ds.fastq.gz"]
      }]
    },
    {
      "dataset_id": "TESTFASTQ2",
      "sample_name": "HG02642",
      "library_name": "SRR580946",
      "sequencing_center": "1000-Genomes",
      "filepaths": [{
        "flowcell_name": "20121211",
        "fastq_r1_locations": ["tests/data/SRR580946_1.ds.fastq.gz"],
        "fastq_r2_locations": ["tests/data/SRR580946_2.ds.fastq.gz"]
      }]
    }
  ]
}
```

This might seem overwhelming, but let's look at the top-level structures first:

```
{                                                                          ①
  "PairedFastqsToUnmappedCram.batch_info": [                               ②
   ...
  ]
}
```

① The top level of the file is a JSON object
② The next level down ("PairedFastqsToUnmappedCram.batch_info") is a list.

This workflow specifies the file inputs using the `PairedFastqsToUnmappedCram.batch_info` object, which is a *list*.

Each sample in the `PairedFastqsToUnmappedCram.batch_info` list is its own object:

```
  "PairedFastqsToUnmappedCram.batch_info": [
    {
      "dataset_id": "TESTFASTQ1",
      "sample_name": "HG02635",
      "library_name": "SRR581005",
      "sequencing_center": "1000-Genomes",
      "filepaths": [{
        "flowcell_name": "20121211",
        "fastq_r1_locations": ["tests/data/SRR581005_1.ds.fastq.gz"],
        "fastq_r2_locations": ["tests/data/SRR581005_2.ds.fastq.gz"]
      }]
    },
    ....
```

Because we are aligned paired-end data, notice there are two keys, `fastq_r1_locations` and `fastq_r2_locations`.

12 Week 4: Workflows

12.1 Learning Objectives

- **Execute** a workflow on the Fred Hutch Cluster
- **Modify** a workflow and **test** it on a compute node
- **Utilize** a container and **test** inside it on a compute node

12.2 Getting today's exercises

In your `bash_for_bio/` folder try doing a `git pull`. Or

copy these files into your own `bash_for_bio` directory:

```
cp /home/tladera2/bash_for_bio/scripts/week4/* ~/bash_for_bio/scripts/week4/
```

12.3 Running Workflows

So far, we've been batch processing using job arrays in SLURM. However, when you graduate to processing in multiple steps, you should consider using a *workflow manager* for processing files.

A workflow manager will take a workflow, which can consist of multiple tasks and allow you to batch process files.

A good workflow manager will allow you to:

- Restart failed subjobs in the workflow
- Allow you to customize where intermediate and final outputs go
- Swap and customize modules in your workflow
- Adapt to different computing architectures (HPC/cloud/etc)

Many bioinformaticists have used workflow managers to process and manage hundreds or thousands of files at a time. They are well worth the extra effort it takes to learn.

Here is an overview of some of the common bioinformatics workflow managers. We will be using `miniWDL`, which runs on WDL files.

Manager Software	Workflow Formats	Notes
Cromwell	WDL/CWL	Made for HPC Jobs
Sprocket	WDL	Made for HPC Jobs
MiniWDL	WDL	Used for local testing of workflows
DNANexus	WDL/CWL	Used for systems such as AllOfUs
Nextflow	`.nf` files	Owned by seqera
Snakemake	make files	

> **MiniWDL is mostly for demonstration**
>
> MiniWDL is written to run locally on your own computer for testing. We don't recommend that you run it on `rhino` for work in general. For one, it is not set up to take advantage of SLURM.
> Cromwell is the suggested way to run workflows on `rhino`. We have been working on a web app called PROOF, that will let you manage your workflows, look at individual jobs, and launches a Cromwell server.

12.3.1 Specifying File Inputs

To run a WDL workflow, you will need to specify the *inputs* for that workflow. These inputs can be reference genomes, the files we want to process, and any other information the workflow requires.

Our WDL workflow specifies the following inputs:

```
input {
    File ref_fasta
    File ref_bwt
    File ref_pac
    File ref_amb
    File ref_ann
    File ref_sa
    File fastq_r1
    String name
  }
```

Note that most of the entries in this input have to do with the reference, including the files that are generated by `bwa index`. This is because `bwa mem` needs to have these files accessible, even though our `bwa mem` script only runs with the `ref_fasta` input.

Here is an example of inputs for our workflow of interest.

```
#| filename: scripts/week4/input.json
{
  "bwa_samtools_workflow.ref_fasta": "/shared/biodata/reference/iGenomes/Homo_sapiens/UCSC/hg
  "bwa_samtools_workflow.ref_bwt": "/shared/biodata/reference/iGenomes/Homo_sapiens/UCSC/hg19
  "bwa_samtools_workflow.ref_pac": "/shared/biodata/reference/iGenomes/Homo_sapiens/UCSC/hg19
  "bwa_samtools_workflow.ref_amb": "/shared/biodata/reference/iGenomes/Homo_sapiens/UCSC/hg19
  "bwa_samtools_workflow.ref_ann": "/shared/biodata/reference/iGenomes/Homo_sapiens/UCSC/hg19
  "bwa_samtools_workflow.ref_sa": "/shared/biodata/reference/iGenomes/Homo_sapiens/UCSC/hg19/
  "bwa_samtools_workflow.fastq_r1": "./data/CALU1_combined_final.fastq",
  "bwa_samtools_workflow.name": "CALU1_combined_final"
}
```

Input Files as JSON

If you want to work with the JSON format (Section 11.2), there is a trick to generating a template `.json` file for a workflow.

```
miniwdl input-template scripts/week4/ww-bwa-samtools.wdl > ww-bwa-inputs.json
```

This will generate a file called `ww-bwa-inputs.json` that contains all of the inputs:

```
{
  "bwa_samtools_workflow.ref_fasta": "File",
  "bwa_samtools_workflow.ref_bwt": "File",
  "bwa_samtools_workflow.ref_pac": "File",
  "bwa_samtools_workflow.ref_amb": "File",
  "bwa_samtools_workflow.ref_ann": "File",
  "bwa_samtools_workflow.ref_sa": "File",
  "bwa_samtools_workflow.fastq_r1": "File",
  "bwa_samtools_workflow.name": "String"
}
```

This can be a good head start to making your `.json` files.
For Cromwell, there is an additional utility called `womtools` that will generate a template file from a workflow file.

12.3.2 Setting up for running a WDL workflow

Say someone has given us a WDL file - how do we set it up to run on our own data?

We'll use `miniWDL` to run our WDL workflow. It is accessible via the `cirro` module.

```
module load cirro
module load Apptainer/1.1.6
export MINIWDL__SCHEDULER__CONTAINER_BACKEND=singularity
```

Because Docker is not available on `rhino`, we need to set the container backend to be `singularity` (the original name for Apptainer). This will enable MiniWDL to launch Docker containers.

If we run

```
which miniwdl
```

We will get the repsonse:

```
/app/software/cirro/1.7.0-foss-2024a/bin/miniwdl
```

Now we're set up to run our WDL workflow.

12.3.3 Running a WDL workflow

We'll use `miniwdl run` to execute our workflow. Again, in our `bash_for_bio` directory:

```
miniwdl run -i scripts/week4/input.json scripts/week4/ww-bwa-samtools.wdl
```

```
2025-11-03 12:40:35.106 miniwdl-run no configuration file found
2025-11-03 12:40:35.998 wdl.w:bwa_samtools_workflow workflow start :: name: "bwa_samtools_wo
bwa-samtools.wdl", line: 2, column: 1, dir: "/home/tladera2/bash_for_bio/20251103_124035_bwa_
2025-11-03 12:40:36.015 wdl.w:bwa_samtools_workflow miniwdl :: version: "v1.13.0", uname: "L
213-generic #224-Ubuntu SMP Mon Jun 19 13:30:12 UTC 2023 x86_64"
2025-11-03 12:40:36.015 wdl.w:bwa_samtools_workflow task thread pool initialized :: task_con
2025-11-03 12:40:36.049 wdl.w:bwa_samtools_workflow visit :: node: "decl-
fastq_r1", values: {"fastq_r1": "/home/tladera2/bash_for_bio/data/CALU1_combined_final.fastq
2025-11-03 12:40:36.049 wdl.w:bwa_samtools_workflow visit :: node: "decl-
name", values: {"name": "CALU-test"}
```

```
[....]

2025-11-03 12:40:57.915 wdl.w:bwa_samtools_workflow finish :: job: "call-
samtools_process"
2025-11-03 12:40:57.916 wdl.w:bwa_samtools_workflow output :: job: "call-
samtools_process", values: {"sorted_bai": "/home/tladera2/bash_for_bio/20251103_124035_bwa_sa
samtools_process/out/sorted_bai/CALU-test_sorted.bam.bai", "sorted_bam": "/home/tladera2/bash
samtools_process/out/sorted_bam/CALU-test_sorted.bam"}
2025-11-03 12:40:57.916 wdl.w:bwa_samtools_workflow visit :: node: "output-
sorted_bai", values: {"sorted_bai": "/home/tladera2/bash_for_bio/20251103_124035_bwa_samtools
samtools_process/out/sorted_bai/CALU-test_sorted.bam.bai"}
2025-11-03 12:40:57.916 wdl.w:bwa_samtools_workflow visit :: node: "output-
sorted_bam", values: {"sorted_bam": "/home/tladera2/bash_for_bio/20251103_124035_bwa_samtools
samtools_process/out/sorted_bam/CALU-test_sorted.bam"}
2025-11-03 12:40:57.916 wdl.w:bwa_samtools_workflow visit :: node: "outputs", values: {"sorte
samtools_process/out/sorted_bai/CALU-test_sorted.bam.bai", "sorted_bam": "/home/tladera2/bash
samtools_process/out/sorted_bam/CALU-test_sorted.bam"}
2025-11-03 12:40:57.942 wdl.w:bwa_samtools_workflow done
```

Success! We got our WDL to run. Let's look at the output.

12.3.4 Exploring the WDL output

The output of our WDL workflow is as follows:

```
output {
  File sorted_bam = samtools_process.sorted_bam
  File sorted_bai = samtools_process.sorted_bai
}
```

So we should expect a `sorted_bam` and a `sorted_bai` file as outputs. So where are they?

MiniWDL will generate a folder per workflow run. You can always see the latest workflow run by referencing `_LAST/`:

```
ls -l _LAST/
```

```
total 248
drwxrwx--- 4 tladera2 g_tladera2   325 Nov  3 12:40 call-bwa_align
drwxrwx--- 4 tladera2 g_tladera2   325 Nov  3 12:40 call-samtools_process
-rw-rw---- 1 tladera2 g_tladera2   911 Nov  3 12:40 inputs.json
```

```
drwxrwx--- 4 tladera2 g_tladera2    56 Nov  3 12:40 out
-rw-rw---- 1 tladera2 g_tladera2   293 Nov  3 12:40 outputs.json
-rw-rw---- 1 tladera2 g_tladera2   127 Nov  3 12:40 rerun
drwxrwx--- 2 tladera2 g_tladera2    37 Nov  3 12:40 wdl
-rw-rw---- 1 tladera2 g_tladera2 14348 Nov  3 12:40 workflow.log
```

In the `_LAST/out/` folder we have:

```
ls -l _LAST/out
```

```
drwxrwx--- 2 tladera2 g_tladera2 42 Nov  3 12:40 sorted_bai
drwxrwx--- 2 tladera2 g_tladera2 38 Nov  3 12:40 sorted_bam
```

Which will contain our output files symbolically linked.

12.4 Looking at a WDL workflow

Workflows are divided up into *workflows* and individual *tasks*. There are two tasks in our workflow, which we can tell by the two `calls` in the workflow: `bwa_align` (2) and `samtools_process` (3).

Each workflow and task requires an `input` and `output`.

```
#| eval: false
#| filename: scripts/week4/ww-bwa-samtools.wdl
version 1.0
workflow bwa_samtools_workflow {
  input {                                                                ①
    File ref_fasta
    File ref_bwt
    File ref_pac
    File ref_amb
    File ref_ann
    File ref_sa
    File fastq_r1
    String name
  }
  call bwa_align {                                                       ②
    input:
      reference_bwt = ref_bwt,
```

```
      reference_ann = ref_ann,
      reference_pac = ref_pac,
      reference_amb = ref_amb,
      reference_sa = ref_sa,
      reference_fasta = ref_fasta,
      reads = fastq_r1,
      name = name
  }

  call samtools_process {                                                    ③
    input:
      sam_file = bwa_align.bwa_output,
      name = name
  }
  output {                                                                   ④
    File sorted_bam = samtools_process.sorted_bam
    File sorted_bai = samtools_process.sorted_bai
  }
}
task bwa_align {                                                             ⑤
  ...
}
task samtools_process {                                                      ⑥
  ...
}
```

① Inputs to workflow
② Call `bwa_align` task using inputs
③ Take outputs of `bwa_align` task and process them with `samtools_process`
④ Outputs of workflow
⑤ `bwa_align` task definition
⑥ `samtools_process` task definition.

You can see that the structure of the WDL file is a lot like a shell script. The inputs have particular expected formats, and it calls `bwa_align` first. Once the outputs have been generated from the `bwa_align` task, they are then used as inputs to the `samtools_process` task, which outputs the final outputs of the workflow.

If we look at the *task* definition called `bwa_align` it looks like this:

```
task bwa_align {
  meta {
    description: "Aligns paired-end reads to a reference using BWA-MEM"
```

```
    outputs: {
      bwa_output: "Output SAM file containing aligned reads"
    }
  }
  parameter_meta {
    reference_fasta: "Reference genome FASTA file"
    reads: "FASTQ of forward (R1) reads"
    name: "Sample name for output SAM file"
  }
  input {
    File reference_fasta
    File reference_bwt
    File reference_ann
    File reference_pac
    File reference_amb
    File reference_sa
    File reads
    String name
  }
  # Get name of reference fasta file within 'bwa_index' folder
  String ref_name = reference_fasta
  command <<<                                                          ①
    # BWA alignment using 8 threads
    bwa mem -t 8 "~{ref_name}" "~{reads}" > "~{name}.sam"
  >>>
  output {
    File bwa_output = "~{name}.sam"
  }
  runtime {
    docker: "getwilds/bwa:0.7.17"
    cpu: 8
    memory: "16 GB"
  }
}
```

① Contains the bash code.

12.4.1 How does this connect to shell scripting?

You'll notice that everything between the:

```
command <<<
```

```
>>>
```

Is essentially a shell script, but adapted to work with the WDL inputs.

`miniwdl check`

Miniwdl has a `check` built in that makes sure that your WDL has correct syntax (in fact it does use `shellcheck` to check your bash script portions). You can run it by using:

```
miniwdl check scripts/week4/ww-bwa-samtools.wdl
```

The output of it is:

```
ww-bwa-samtools.wdl
    workflow bwa_samtools_workflow
        call bwa_align
        call samtools_process
    task bwa_align
        (Ln 60, Col 5) UnusedDeclaration, nothing references File reference_bwt
        (Ln 61, Col 5) UnusedDeclaration, nothing references File reference_ann
        (Ln 62, Col 5) UnusedDeclaration, nothing references File reference_pac
        (Ln 63, Col 5) UnusedDeclaration, nothing references File reference_amb
        (Ln 64, Col 5) UnusedDeclaration, nothing references File reference_sa
    task samtools_process
```

In our case, you can see that it is complaining that we aren't "using" the `.bwt` and other index files. This is ok, because we just need to have the files present to run `bwa` even if we aren't directly using them in our script.

12.5 `scatter()`: how we divide up tasks.

We can batch process our `fastq` files in `data/` by using the `scatter` function in WDL. There are two things we need to do to modify our workflow:

1. Accept an `Array` of `File`s called
2. Use `scatter` to iterate over the files by wrapping our two tasks, `bwa_align` and `samtools_process`.

Our `input` definition for the workflow looks like this:

```
input {
```

```
    File ref_fasta
    File ref_bwt
    File ref_pac
    File ref_amb
    File ref_ann
    File ref_sa
    Array[File] fastq_r1
  }
```

Which is identical to our other WDL, but it has a `Array[File]` as an input. The corresponding input in our JSON file looks like this:

```
  "bwa_samtools_workflow.fastq_r1": [
    "data/CALU1_combined_final.fastq",
    "data/HCC4006_final.fastq",
    "data/MOLM13_combined_final.fastq"
  ]
```

Which is a JSON list of the files we want to process.

When we run

```
miniwdl run -i scripts/week4/scatter.json scripts/week4/ww-bwa-samtools-scatter.wdl
```

We get the following output:

```
{
  "dir": "/home/tladera2/bash_for_bio/20251104_101700_bwa_samtools_workflow",
  "outputs": {
    "bwa_samtools_workflow.sorted_bai": [
      "/home/tladera2/bash_for_bio/20251104_101700_bwa_samtools_workflow/out/sorted_bai/0/CAL
      "/home/tladera2/bash_for_bio/20251104_101700_bwa_samtools_workflow/out/sorted_bai/1/HCC
      "/home/tladera2/bash_for_bio/20251104_101700_bwa_samtools_workflow/out/sorted_bai/2/MOL
    ],
    "bwa_samtools_workflow.sorted_bam": [
      "/home/tladera2/bash_for_bio/20251104_101700_bwa_samtools_workflow/out/sorted_bam/0/CAL
      "/home/tladera2/bash_for_bio/20251104_101700_bwa_samtools_workflow/out/sorted_bam/1/HCC
      "/home/tladera2/bash_for_bio/20251104_101700_bwa_samtools_workflow/out/sorted_bam/2/MOL
    ]
  }
}
```

Examining the last workflow run directory,

```
ls -l _LAST/out/sorted_bam/
```

We see that our output has 3 subdirectories:

```
total 120
drwxrwx--- 2 tladera2 g_tladera2 59 Nov  4 10:17 0
drwxrwx--- 2 tladera2 g_tladera2 52 Nov  4 10:17 1
drwxrwx--- 2 tladera2 g_tladera2 60 Nov  4 10:17 2
```

Each of these directories contains a symbolic link to the output files.

12.5.1 More about `scatter`

Here is the `scatter` block in the workflow:

```
  scatter (fq in fastq_r1){                                          ①
    call bwa_align {
        input:
            reference_bwt = ref_bwt,
            reference_ann = ref_ann,
            reference_pac = ref_pac,
            reference_amb = ref_amb,
            reference_sa = ref_sa,
            reference_fasta = ref_fasta,
            reads = fq                                               ②
        }

   call samtools_process {
    input:
      sam_file = bwa_align.bwa_output                                ③
        }
  }
```

① `scatter` over our `Array[File] fastq_r1`. We use similar syntax as for loops to iterate over each file with a placeholder `fq`.
② Utilize `fq` placeholder in the `call bwa_align` task.
③ The rest of the workflow works as usual.

The output section of the workflow looks like this:

```
output {
    Array[File] sorted_bam = samtools_process.sorted_bam
    Array[File] sorted_bai = samtools_process.sorted_bai
  }
```

Note that the `scatter` generates an `Array` of files as well.

12.5.2 For more information

I recommend that you read the Developing WDL Workflows book that we have written for much more information about building your own workflows.

You don't have to start from scratch, either. There is the WILDS WDL library that has a number of different bioinformatics tasks that are already built for use in your workflows.

12.6 Working with file manifests in WDL

Another strategy you can use to cycle through a list of files is to use a file manifest to process a list of files.

There is an example workflow that shows how to work with a file manifest. This can be helpful for those who aren't necessarily good at working with JSON.

This workflow has a single input, which is the location of the file manifest. It will then cycle through the manifest, line by line.

This is what the file manifest contains. Notice there are three named columns for this file: `sampleName`, `bamLocation`, and `bedLocation`.

sampleName	bamLocation	bedLocation
smallTestData	/fh/fast/paguirigan_a/pub/ReferenceDataSets/workflow_testing_data/WDL/unpaired-panel-consensus-variants-human/smallTestData.unmapped.bam	/fh/fast/paguirigan_a/pub/ReferenceDataSets/reagent_specific_data/sequencing_panel_bed/TruSightMyeloid-Illumina/trusight-myeloid-amplicon-v4-track_interval-liftOver-hg38.bed

sampleName	bamLocation	bedLocation
smallTestData-reference	/fh/fast/paguirigan_a/pub/ReferenceDataSets/workflow_testing_data/WDL/paired-panel-consensus-variants-human/smallTestData-reference.unmapped.bam	/fh/fast/paguirigan_a/pub/ReferenceDataSets/reagent_specific_data/sequencing_panel_bed/TruSight-Myeloid-Illumina/trusight-myeloid-amplicon-v4-track_interval-liftOver-hg38.bed

12.7 Where Next?

Now that you understand the basics of working with Bash and WDL, you are now ready to try testing and profiling your jobs.

13 Week 4 Exercises

13.1 In class exercises:

1. Setup `miniwdl` to run:

```
module load cirro
module load Apptainer/1.1.6
export MINIWDL__SCHEDULER__CONTAINER_BACKEND=singularity
```

2. Examine `scripts/week4/input.json`. Run `scripts/week4/ww-bwa-samtools.wdl` using `scripts/week4/input.json` (run in `bash_for_bio`):

```
miniwdl run -i scripts/week4/input.json scripts/week4/ww-bwa-samtools.wdl
```

Examine the output directory:

```
ls -l _LAST/
```

Look at the outputs:

```
ls -l _LAST/out
```

3. Examine `scripts/week4/scatter.json`. Run `scripts/week4/ww-bwa-samtools-scatter.wdl` using `scripts/week4/scatter.json`

```
miniwdl run -i scripts/week4/scatter.json scripts/week4/ww-bwa-samtools-
scatter.wdl
```

13.2 Homework Exercises

- No follow up homework - I suggest you take a look at the WILDS WDL library and identify any tasks you might want to use in your work.
- Keep an eye out for the Workflows Data Deep Dive next Winter Quarter for more hands-on WDL experience.

14 Testing Scripts

14.1 The cardinal rule of testing scripts

You should identify a file that you will take through your workflow or SLURM job that is representative of your other files. One approach is to pick the largest file you will process - that gives you a upper bound on memory usage and CPU utilization.

Then you are going to test your workflow on that representative file.

14.1.1 Profiling Jobs using SLURM

There are two ways of profiling your job:

- To Profile a running job, use `sstat <JOBID> -o alloccpus, allocnodes, avecpu, averss`
- To profile a job that has finished, use `sacct -j <JOBID> -o jobid,alloccpus, allocnodes, avecpu, averss`

For example, I can look at the usage of the `miniwdl run` I just ran using:

```
sacct -j 41381212 -o jobid,jobname,alloccpus,allocnodes,avecpu,averss
```

```
JobID           JobName  AllocCPUS AllocNodes     AveCPU     AveRSS
------------ ---------- ---------- ---------- ---------- ----------
41381212_1   run_sbatc+          1          1
41381212_1.+      batch          1          1   00:00:00      7724K
41381212_1.+     extern          1          1   00:00:00      1168K
41381212_2   run_sbatc+          1          1
41381212_2.+      batch          1          1   00:00:00      3236K
41381212_2.+     extern          1          1   00:00:00      1156K
41381212_3   run_sbatc+          1          1
41381212_3.+      batch          1          1   00:00:00      7596K
41381212_3.+     extern          1          1   00:00:00      1160K
```

- `JobID` is the subjob id per *step*

- `JobName` is the step run per subjob
- `AllocCPUS` is the number of CPUs assigned to the task
- `AllocNodes` is the number of Nodes assigned to the task
- `AveCPU` is the average CPU time, defined as the compute time * the number of cores
- `AveRSS` is the total memory used in that step of the process

So, when you run a single task on the job, you will understand whether you are using the resources of each node properly.

More information on profiling: https://csc-training.github.io/csc-env-eff/hands-on/batch_resources/tutorial_sacct_and_seff.html

14.2 Developing and Testing scripts interactively

One of the hard things to understand is what can be run on a compute node versus the head node, and what file systems are accessible via a compute node.

A lot of the issues you might have is because you need to understand the mental model of how cluster computing works. And the best way to understand that is to test your code on a compute node.

Let's explore how we can do that. You should also review the material about using `screen` (Section 16.7).

14.2.1 Testing code on a compute node

Fred Hutch users have the advantage of `grabnode`, which is a custom command that lets you request an interactive instance of a compute node. (Non-FH folks can usually request this with the `-it` flag for `srun`)

Why would you want to do this? A good part of this is about testing software and making sure that your paths are correct.

Don't rely on `grabnode`/interactive mode for your batch work

We often see users that will request a multicore node with higher memory, and do their processing on that node.
This doesn't take advantage of all of the machines that are available on the cluster, and thus is a suboptimal way to utilize the cluster.
When you are doing interactive analysis, such as working in JupyterLab or RStudio, that is a valid way to work. But when you have tasks you can *scatter* amongst many nodes, requesting a high-spec node isn't a great way to optimally achieve things.
The other disadvantage is that you may be waiting a very long time to get that multicore

node, whereas if you batch across a bunch of nodes, you will get your work done much faster.

14.2.2 Grabbing an interactive shell on a worker

When you're testing code that's going to run on a worker node, you need to be aware of what the worker node sees.

It's also important in estimating how long our tasks are going to run since we can test how long a task runs for a representative dataset.

For FH Users: `grabnode`

On the FH system, we can use a command called `grabnode`, which will let us request a node. It will ask us for our requirements (numbers of cores, memory, etc.) for our node.

```
tladera2@rhino01:~$ grabnode
```

`grabnode` will then ask us for what kind of instance we want, in terms of CPUs, Memory, and GPUs. Here, I'm grabbing a node with 8 cores, 8 Gb of memory, using it for 1 day, and no GPU.

```
How many CPUs/cores would you like to grab on the node? [1-36] 8
How much memory (GB) would you like to grab? [160] 8
Please enter the max number of days you would like to grab this node: [1-
7] 1
Do you need a GPU ? [y/N]n

You have requested 8 CPUs on this node/server for 1 days or until you type exit.

Warning: If you exit this shell before your jobs are finished, your jobs
on this node/server will be terminated. Please use sbatch for larger jobs.

Shared PI folders can be found in: /fh/fast, /fh/scratch and /fh/secure.

Requesting Queue: campus-new cores: 8 memory: 8 gpu: NONE
srun: job 40898906 queued and waiting for resources
```

After a little bit, you'll arrive at a new prompt:

```
(base) tladera2@gizmok164:~$
```

Now you can test your batch scripts, in order to make sure your file paths are correct. It is also helpful in profiling your job.
If you're doing interactive analysis that is going to span over a few days, I recommend that you use `screen` or `tmux`.

For Other HPC systems

On a SLURM system, the way to open interactive shells on a node has changed. Check your version first:

```
srun --version
```

If you're on a version before 20.11, you can use `srun -i --pty bash` to open an interactive terminal on a worker:

```
srun -i --pty bash
```

If the version is past 20.11, we can open an interactive shell on a worker with `salloc`.

```
salloc bash
```

Remember `hostname`

When you are doing interactive analysis, it is easy to forget in which node you're working in. Just as a quick check, I use `hostname` (Section 16.1) to remind myself whether I'm in `rhino`, `gizmo`, or within an apptainer container.

15 Appendix: Configuring your Shell

15.1 `.bashrc`: Where do I put my configuration?

There is a file in your home directory called `.bashrc`. This is where you can customize the way the Bash shell behaves.

There are 2 things you should know how to set:

- Aliases
- Environment Variables, especially `$PATH`

15.2 Aliases

Aliases are shortcuts for commands. You can specify them using `alias` as a line in your `.bashrc` file:

```
alias ll='ls -la'
```

We are defining an alias called `ll` that runs `ls -la` (long listing for directory for all files) here. Once

Some people even add aliases for things they mistype frequently.

15.3 Environment Variables

Environment variables are variables which can be seen globally in the Linux (or Windows) system across executables.

You can get a list of all set environment variables by using the `env` command. Here's an example from my own home system:

```
env
```

```
SHELL=/bin/bash
NVM_INC=/home/tladera2/.nvm/versions/node/v21.7.1/include/node
NAME=2QM6TV3
PWD=/home/tladera2
LOGNAME=tladera2
[....]
```

One common environment variable you may have seen is `$JAVA_HOME`, which is used to find the Java Software Development Kit (SDK). (I usually encounter it when a software application yells at me when I haven't set it.)

You can see whether an environment variable is set using `echo`, such as

```
echo $PATH
```

```
/home/tladera2/.local/bin:/home/tladera2/gems/bin:/home/tladera2/.nvm/versions/node/v21.7.1/
```

15.3.1 Setting Environment Variables

In Bash, we use the `export` command to declare an environment variable. For example, if we wanted to declare the environment variable `$SAMTOOLS_PATH` we'd do the following:

```
# works: note no spaces
export SAMTOOLS_PATH="/home/tladera2/miniconda/bin/"
```

One thing to note is that spacing matters when you declare environment variables. For example, this won't declare the `$SAMTOOLS_PATH` variable:

```
# won't work because of spaces
export SAMTOOLS_PATH = "/home/tladera2/miniconda/bin/"
```

Another thing to note is that we declare environment variables differently than we use them. If we wanted to use `SAMTOOLS_PATH` in a script, we use a dollar sign (`$`) in front of it:

```
${SAMTOOLS_PATH}/samtools view -c $input_file
```

In this case, the value of `$SAMTOOLS_PATH` will be expanded (substituted) to give the overall path:

```
/home/tladera2/miniconda/bin/samtools view -c $input_file
```

15.3.2 A Very Special Environment Variable: `$PATH`

The most important environment variable is the `$PATH` variable. This variable is important because it determines where to search for software executables (also called binaries). If you have softwware installed by a package manager (such as `miniconda`), you may need to add the location of your executables to your `$PATH`.

We can add more directories to the `$PATH` by appending to it. You might have seen the following bit of code in your `.bashrc`:

```
export PATH=$PATH:/home/tladera2/samtools/
```

In this line, we are adding the path `/home/tladera2/samtools/` to our `$PATH` environment variable. Note that how we refer to the `PATH` variable is different depending on which side the variable is on of the equals sign.

> **Order matters in your `$PATH`**
>
> The order of directories is the order in which `bash` will look for executables. If you install software yourself on `gizmo` (using `make` or such), you should *prepend* the binary directory with your software to the `$PATH`:
>
> ```
> export PATH=/home/tladeras/samtools/:$PATH
> ```

TLDR: We declare the variable using `export PATH` (no dollar sign) and we append or prepend to this variable using `$PATH` (with dollar sign). This is something that trips me up all the time.

> **For FH Users**
>
> In general, when you use environment modules on `gizmo`, you do not need to modify your `$PATH` variable. You mostly need to modify it when you are compiling executables so that the system can find them. Be sure to use `which` to see where the environment module is actually located:
> `which samtools`

15.3.3 Making your own environment variables

One of the difficulties with working on a cluster is that your scripts may be in one filesystem (`/home/`), and your data might be in another filesystem (`/fh/fast/`). And it might be recommended that you transfer over files to a faster-access filesystem (`/fh/temp/`) to process them.

You can set your own environment variables for use in your own scripts. For example, we might define a `$TCR_FILE_HOME` variable:

```
export TCR_FILE_HOME=/fh/fast/my_tcr_project/
```

to save us some typing across our scripts. We can use this new environment variable like any other existing environment variable:

```
#!/bin/Bash
export my_file_location=$TCR_FILE_HOME/fasta_files/
```

> **`.bashrc` versus `.bash_profile`**
>
> Ok, what's the difference between `.bashrc` and `.bash_profile`?
> The main difference is when these two files are sourced. `bash_profile` is used when you do an interactive login, and `.bashrc` is used for non-interactive shells.
> `.bashrc` should contain the environment variables that you use all the time, such as `$PATH` and `$JAVA_HOME` for example.
> You can get the best of both worlds by including the following line in your `.bash_profile`:
>
> ```
> source ~/.bashrc
> ```
>
> That way, everything in the `.bashrc` file is loaded when you log in interactively.

16 Miscellaneous

16.1 `hostname` What Machine am I on?

One of the most confusing things about working on HPC is that sometimes you have a shell open on the head node, but oftentimes, you are on a worker node.

Your totem for telling which node you're in is `hostname`, which will give you the host name of the machine you're on.

For example, if I used `grabnode` to grab a `gizmo` node for interactive work, I can check which node I'm in by using:

```
hostname
```

```
gizmok164
```

If you're confused about which node you're in, remember `hostname`. It will save you from making mistakes, especially when using utilities like `screen`.

16.1.1 Try it out

After logging into `rhino`, try running `hostname`. What host are you on?

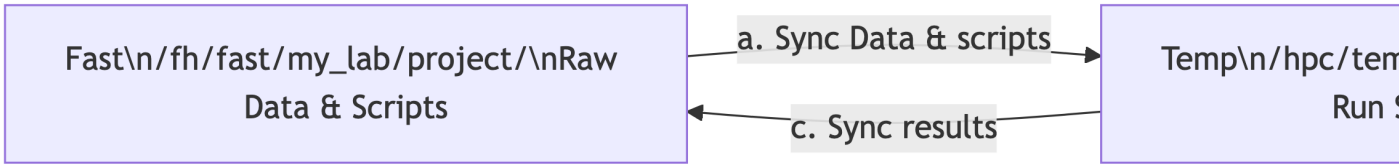
16.2 Workflows

16.2.1 One Workflow: `/fh/fast/` and `/hpc/temp/`

One approach is to have your scripts also live in your project folder in `fast`. Then, you can sync the project in `/fh/fast/` over to `/hpc/temp/`, run the scripts in `/hpc/temp/`, and then sync the two folders again. You can do the file sync'ing in both directions with Motuz, which has its own advantages.

If you want to go this route, you should think about using a Folder Based Workflow (Section 16.10), where everything lives in a folder.

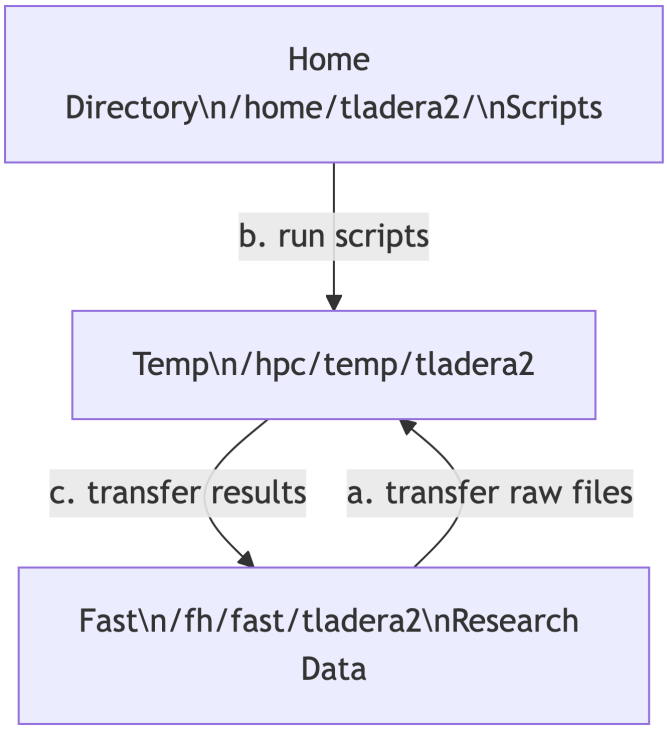
Another thing to consider is to have a backup of the scripts that is either on your own machine or in GitHub. You can do this by using your `.gitignore` to exclude the data and results.



16.2.2 Another Approach

Below is a a diagram with another way to work with these multiple filesystems.

a. We transfer the raw files to be processed from `/fh/fast/` to our directory `/fh/temp/`. For example, a set of `.bam` files.
b. We run our scripts from `/home/`, on the raw files in `/fh/temp/` and produce results in `/fh/temp/`.
c. We transfer our results from `/fh/temp/` to `/fh/fast/`.



16.3 Quoting and Escaping Filenames in Bash

One point of confusion is when do you quote things in Bash? When do you use single quotes (`'`) versus double-quotes (`"`)? When do you use \ to escape characters?

Let's talk about some quoting rules in Bash. I've tried to make things as simplified and generalized as possible, rather than stating all of the rules for each quote.

1. If you have spaces in a filename, use double quotes (`"chr 1.bam"`)
2. If you have a single quote in the filename, use double quotes to wrap it (`"ted's file.bam"`)
3. Only escape characters when necessary - if you can solve a problem with quotes, use them
4. If you need to preserve an escaped character, use single quotes

Let's go over each of these with an example.

16.3.1 If you have spaces in a filename, use double quotes (Most common)

For example, if your filename is `chr 1 file.bam`, then use double quotes in your argument

```
samtools view -c "chr 1 file.bam"
```

16.3.2 If you have a single quote in the name, use double quotes to wrap it (less common)

Say you have a file called `ted's new file.bam`. This can be a problem when you are calling it, especially because of the single quote.

In this case, you can do this:

```
samtools view -c "ted's new file.bam"
```

16.3.3 Only escape characters when necessary (less common)

There are a number of special characters (such as Tab, and Newline) that can be specified as escape characters. In double quotes, characters such as `$` are signals to Bash to expand or evaluate code.

Say that someone had a `$` in their file name such as `Thi$file is money.bam`

How do we refer to it? We can escape the character with a backslash \:

```
samtools view -c "Thi\$file is money.bam"
```

The backslash is a clue to Bash that we don't want variable expansion in this case. Without it, bash would look for a variable called `$file`.

16.3.4 If you need to preserve an escaped character, use single quotes (least common)

This is rarely used, but if you need to keep an escaped character in your filename, you can use single quotes. Say we have a filename called `Thi\$file.bam` and you need that backslash in the file name (btw, please don't do this), you can use single quotes to preserve that backslash:

```
samtools view -c 'Thi\$file.bam'
```

Again, hopefully you won't need this.

16.3.5 For More Info

https://www.grymoire.com/Unix/Quote.html#uh-3

> **What about backticks?**
>
> Backticks (`) are an old way to do command evaluation in Bash. For example, if we run the following on the command-line:
>
> ```
> echo "there are `ls -l | wc -l` files in this directory"
> ```
>
> Will produce:
>
> ```
> there are       36 files in this directory
> ```
>
> Their use is deprecated, so you should be using `$()` in your command evaluations instead:
>
> ```
> echo "there are $(ls -l | wc -l) files in this directory"
> ```

> **What about X use case?**
>
> There are a lot of rules for Bash variable expansion and quoting that I don't cover here. I try to show you a way to do things that work in multiple situations on the cluster.
> That's why I focus on double quotes for filenames and `${}` for variable expansion in general. They will work whether your Bash script is on the command line or in an App,

or in WDL.

16.4 Using pipes: STDIN, STDOUT, STDERR

We will need to use pipes to chain our commands together. Specifically, we need to take a command that generates a list of files on the cluster shared filesystem, and then spawns individual jobs to process each file. For this reason, understanding a little bit more about how pipes (`|`) work in Bash is helpful.

If we want to understand how to chain our scripts together into a pipeline, it is helpful to know about the different streams that are available to the utilities.

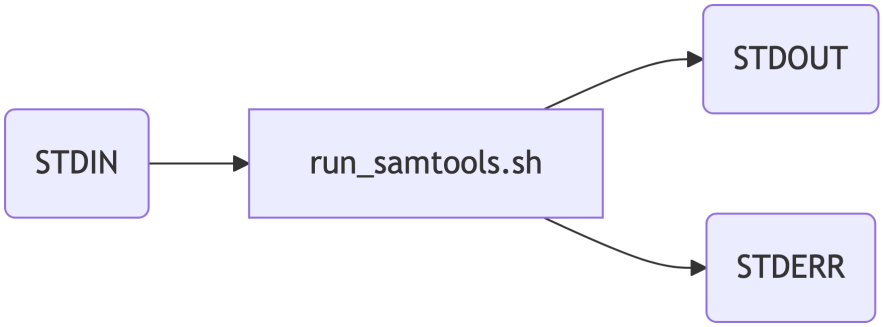


Figure 16.1: Inputs/outputs to a script

Every script has three streams available to it: Standard In (STDIN), Standard Out (STDOUT), and Standard Error (STDERR) (Figure 16.1).

STDIN contains information that is directed to the input of a script (usually text output via STDOUT from another script).

Why do these matter? To work in a Unix pipeline, a script must be able to utilize STDIN, and generate STDOUT, and STDERR.

Specifically, in pipelines, STDOUT of a script (here it's `run_samtools`) is directed into STDIN of another command (here `wc`, or word count)

We will mostly use STDOUT in our bash scripts, but STDERR can be really helpful in debugging what's going wrong.

> **Why this is important on the Cluster**
>
> We'll use pipes and pipelines not only in starting a bunch of jobs using batch scripting on our home computer, but also when we are processing files within a job.

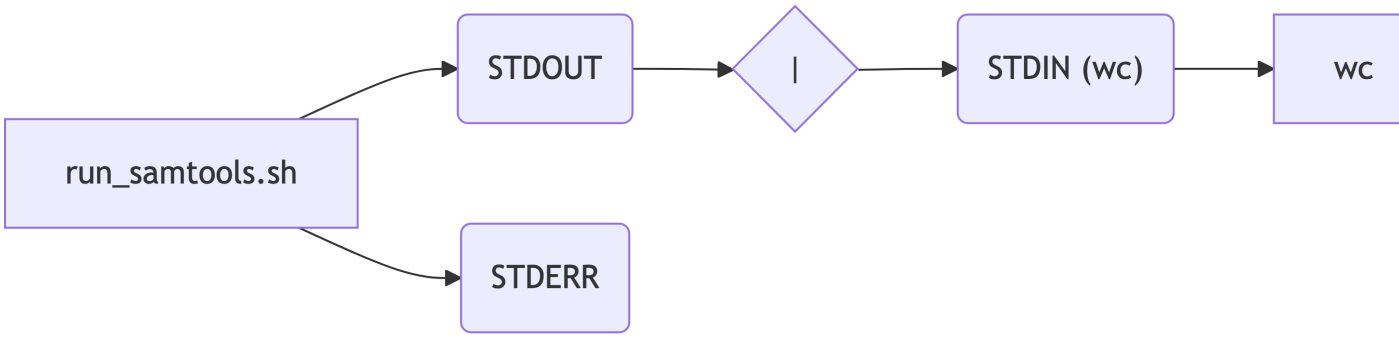


Figure 16.2: Piping a script `run_samtools.sh` into another command (`wc`)

16.4.1 For more info about pipes and pipelines

https://swcarpentry.github.io/shell-novice/04-pipefilter/index.html https://datascienceatthecommandline.com/2e/chapter-2-getting-started.html?q=stdin#combining-command-line-tools

16.5 `basename` can be very handy when on workers

If we are processing a bunch of files on a worker, we need a way to get the bare filename from a path. We will take advantage of this when we run process multiple files on the worker.

For example:

```
basename /mnt/project/worker_scripts/srun-script.sh
```

This will return:

```
srun-script.sh
```

Which can be really handy when we name our outputs. This command is so handy it is used in WDL.

16.6 Advanced Batch Processing: Iterating using `xargs`

A really common pattern is taking a delimited list of files and doing something with them. We can do some useful things such as seeing the first few lines of a set of files, or doing some sort of processing with the set of jobs.

> **Don't `xargs` for HPC jobs**
>
> You might be tempted to use `xargs` with `srun` to work on a bunch of files. It's worth trying once so you can see the mechanics of how jobs are processed.
> In general, I don't recommend it in practice because if you spawn 1000 jobs using `xargs`, there's no real mechanism to terminate that 1000 jobs, except one by one. With `sbatch`, all your jobs in batch mode run as *subjobs*, which means you can terminate the parent job to terminate all of the subjobs.
> Again, this is a good reason to use a workflow runner in your day to day work. You don't have to worry about jobs and subjobs. It takes a little setup, but it will make your life easier in general.

Let's start out with a list of files:

```
source ~/.bashrc #| hide_line
ls data/*.sh
```

```
data/batch-on-worker.sh
```

Now we have a list of files, let's look at the first few lines of each of them, and print a separator `---` for each.

```
#| filename: scripting-basics/xargs_example.sh
source ~/.bashrc #| hide_line
ls data/*.sh | xargs -I% sh -c 'head %; echo "\n---\n"'
```

```
#!/bash/bin

cmd_to_run="ls *.vcf.gz | xargs -I% sh -c "bcftools stats % > %.stats.txt"

dx run swiss-army-knife \
  -iin="data/chr1.vcf.gz" \
  -iin="data/chr2.vcf.gz" \
  -iin="data/chr3.vcf.gz" \
  -icmd=${cmd_to_run}
---
dx find data --name "*.bam" --brief
---
```

Let's take this apart piece by piece.

`xargs` takes an `-I` argument that specifies a placeholder. In our case, we are using `%` as our placeholder in this statement.

We're passing on each filename from `ls` into the following code:

```
sh -c 'head %; echo "---\n"'
```

The `sh -c` opens a subshell so that we can execute our command for each of the files in our list. We're using `sh -c` to run:

```
'head %; echo "---\n"'
```

So for our first file, `01-scripting-basics.qmd`, we are substituting that for `%` in our command:

```
'head hpc-basics.qmd; echo "---\n"'
```

For our second file, `hpc-basics.qmd`, we would substitute that for the `%`:

```
'head hpc-basics.qmd; echo "---\n"'
```

Until we cycle through all of the files in our list.

16.6.1 The Basic `xargs` pattern

Figure 16.3: Basics of using `xargs` to iterate on a list of files

As you cycle through lists of files, keep in mind this basic pattern (Figure 16.3):

```
ls <wildcard> | xargs -I% sh -c "<command to run> %"
```

Test Yourself

How would we modify the below code to do the following?

1. List only `.json` files in our `data/` folder using `ls`
2. Use `tail` instead of `head`

```
ls *.txt | xargs -I% sh -c "head %; echo '---\n'"
```

Answer

```
ls data/*.json | xargs -I% sh -c "tail %; echo '---\n'"
```

Why this is important on HPC

We can use `xargs` to execute small batch jobs on a small number of files. This especially becomes powerful on the cluster when we use `ls` to list files in our HPC project.
Note that as we *graduate* to workflow tools like WDL/Nextflow, there are other mechanisms for running jobs on multiple files (such as WDL/Cromwell) that we should move to.
Trust me; you don't want to have to handle iterating through a huge directory and handling when routines give an error, or your jobs get interrupted. Rerunning and resuming failed jobs are what workflow runner tools excel at.

16.6.2 For more information

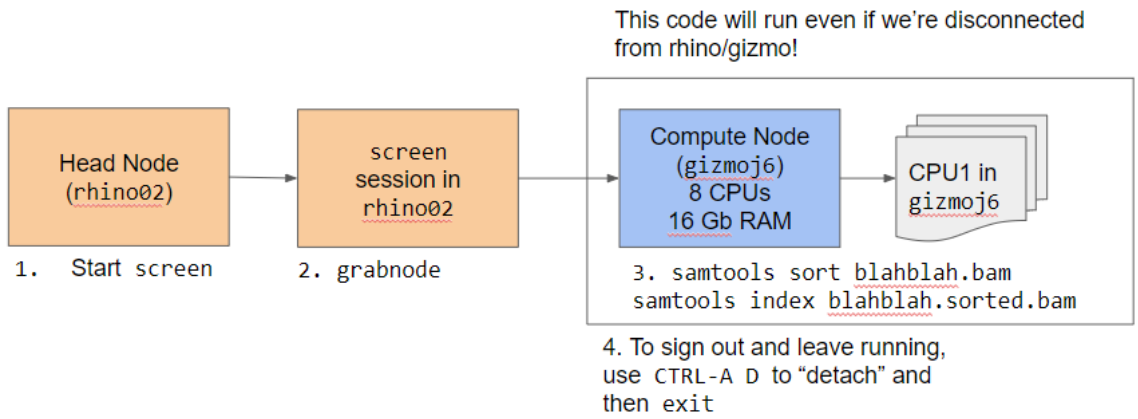
https://www.baeldung.com/linux/xargs-multiple-arguments

16.7 `screen` or `tmux`: keep your session open

https://youtu.be/fYWszwc-LXM

Oftentimes, when you are running something interactive on a system, you'll have to leave your shell open. Otherwise, your running job will terminate.

You can use `screen` or `tmux`, which are known as window managers, to keep your sessions open on a remote machine. We'll talk about `screen`.

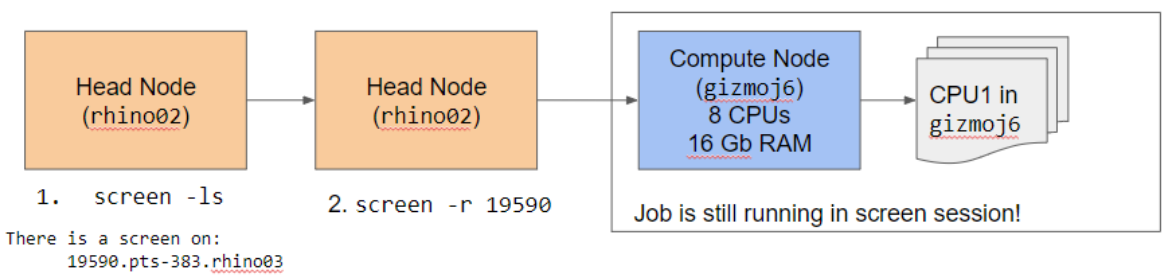


`screen` works by starting a new bash shell. You can tell this because your bash prompt will change.

The key of working remotely with `screen` is that you can then request an `hpc` node.

For FH users, you can request a `gizmo` node using `grabnode`. We can then check we're on the `gizmo` node by using `hostname`.

If we have something running on this node, we can keep it running by *detaching* the screen session. Once we are detached, we should check that we're back in `rhino` by using `hostname`. Now we can log out and our job will keep running.



If we need to get back into that `screen` session, we can use:

```
screen -ls
```

To list the number of sessions:

```
There is a screen on:
        37096.pts-321.rhino01   (05/10/2024 10:21:54 AM)        (Detached)
```

```
1 Socket in /run/screen/S-tladera2.
```

Once we've found the id for our screen session (in this case it's `37096`), we can *reattach* to the screen session using:

```
screen -r 37096
```

And we'll be back in our screen session! Handy, right?

> **For FH Users**
>
> Note that if you logout from `rhino`, you'll need to log back into the same `rhino` node to access your screen session.
> For example, if my screen session was on `rhino01`, I'd need to ssh back into `rhino01`, not `rhino02` or `rhino03`. This means you will need to ssh into `rhino01` specifically to get back into your screen session.

16.8 Using file manifests

One approach that I use a lot is using file manifests to process multiple sets of files. Each line of the file manifest will contain all of the related files I need to process.

For example, if I am aligning paired-end reads, then I can have a tab-separated column for the first strand, and a column for the second strand.

```
sample1_1.fq   sample1_2.fq
sample2_1.fq   sample2_2.fq
```

The one trick with using file manifests in bash is that we need to change the what's called the internal field separator (IFS), which specifies how to split up a string with a `for` loop. By default, bash uses an IFS of " " (a space), which means that the for loop will cycle through words (strings separated by spaces), instead of lines.

We can change this behavior by setting the IFS at the beginning of our script:

```
OUTPUT_FOLDER="."
FASTA_INDEX_LOCATION="/shared/biodata/reference/iGenomes/Homo_sapiens/UCSC/hg19/Sequence/BWA
for file in (cat manifest.txt)                                                    ①
IFS=""                                                                            ②
do
```

```
  read1=$(echo $file | awk '{print $1}')                                    ③
  read2=$(echo $file | awk '{print $2}')                                    ④
  bwa mem ${FASTA_INDEX_LOCATION} ${read1} ${read2} > ${OUTPUT_FOLDER}/${read1}.sam
done

unset IFS                                                                   ⑤
```

① Cycle through each line in `manifest.txt`
② Change IFS to be `""` (no space), to process a file line by line.
③ Grab the word in the 1st position and assign it to `$read1`
④ Grab the word in the 2nd position and assign it to `$read2`
⑤ Reset IFS to original behavior.

The line:

```
read1=$(echo $file | awk '{print $1}')
```

Takes some unpacking. The first thing to notice is that we're assigning something into the `$read1` variable.

This part:

```
$(echo $file | awk '{print $1}')
```

puts `$file` through a command called `awk` that lets us extract words of a string by position. In our case, our different parts of the string are separated by tabs.

The `$(  )` part lets us run a *subshell* and then return the value of the output.

This part:

```
'{print $1}'
```

Are instructions to `awk` to output the first word of our string. `awk` is an extremely powerful command, but we don't have enough time to cover it in full. Julia Evans has a great page on it in Bite Size Command Line.

So, when we put it together, the command assigns `$read1` the value of the first position (`sample1_1.fq`), letting us do things with this file path.

We'll see that you can also use file manifests with workflow runners such as Cromwell.

16.9 Removing file extensions with variable expansion

Oftentimes, when you are processing a file, you want to generate a new file name with the same basename, but not the same extension. For example, you might want to generate a `mysamfile.txt` from a file callled `mysamefile.sam`.

You can use `"${filename%.sam}"` to remove the `.sam` from the file name.

```
#/bin/bash
output_file="${1%.sam}.txt"
echo $output_file
```

If you ran this script with

```
./run_output.sh mybamfile.sam
```

You'll get the output:

```
mybamfile.txt
```

This is a very helpful trick. Note that it only works if the string to remove (the file extension) is a suffix at the end of the filepath. There is a similar trick using `#` if you want to remove the prefix of a file name.

16.10 Project/folder based workflows

On a particular machine, using *absolute* paths is safe. However, you do this at the cost of *portability* - code that you write on one machine may not run on another.

If you ever anticipate doing the analysis on a separate machine, using project structures with relative paths is the safest. For example, you may want to move from the on-premise FH system to working with the data in AWS.

Here's one example of putting everything into a single folder:

```
my_project
  data
      chr1.fa.gz
      chr2.fa.gz
      chr3.fa.gz
  results
```

```
   run_workflow.sh
   scripts
       run_bowtie.sh
```

In the above example, our project is named `my_project`, and there are three folders inside it: `data/`, `results/`, and `scripts/`. Our main script for running is `my_project/run_workflow.sh`. Because this script is in the root folder, we can refer to the `data/` folder to process files:

```
./scripts/run_bowtie.sh data/*.fa.gz results/
```

When we run `run_workflow.sh`, it will execute `run_bowtie.sh` on all of the files in `data/`, and save them in `results/`, resulting in the following updated structure.

```
my_project
   data
       chr1.fa.gz
       chr2.fa.gz
       chr3.fa.gz
   results
       chr1.bam
       chr2.bam
       chr3.bam
   run_workflow.sh
   scripts
       run_bowtie.sh
```

You may have seen relative paths such as `../another_directory/` - the `..` means to go up a directory in the file hierarchy, and then look in that directory for the `another_directory/` directory. I try to avoid using relative paths like these.

In general for portability and reproducibility, you will want to use relative paths **within a directory**, and avoid using relative paths like `../../my_folder`, where you are navigating up. In general, use relative paths to navigate down.

> **Why This is Important**
>
> When you start executing scripts, it's important to know where the results go. When you execute SAMtools on a file in `/fh/temp/`, for example, where does the output go? Workflow Runners such as Cromwell and Nextflow will output into certain file structures by default. This can be changed, but knowing the default behavior is super helpful when you don't specify an output directory.

16.10.1 Let's start with WDL Tasks

We will start with the lower level of abstraction in WDL: The *task*.

16.10.2 Anatomy of a Task

```
task build_star_index {
  meta {
      ...
  }

  parameter_meta {
    reference_fasta: "Reference genome FASTA file"
    reference_gtf: "Reference genome GTF annotation file"
    sjdb_overhang: "Length of the genomic sequence around the annotated junction to be used i
    genome_sa_index_nbases: "Length (bases) of the SA pre-indexing string, typically between
    memory_gb: "Memory allocated for the task in GB"
    cpu_cores: "Number of CPU cores allocated for the task"
  }

  input {                                                                                  ①
    File reference_fasta
    File reference_gtf
    Int sjdb_overhang = 100
    Int genome_sa_index_nbases = 14
    Int memory_gb = 64
    Int cpu_cores = 8
  }

  command <<<                                                                              ②
    set -eo pipefail

    mkdir star_index

    echo "Building STAR index..."
    STAR \
      --runMode genomeGenerate \
      --runThreadN ~{cpu_cores} \
      --genomeDir star_index \
      --genomeFastaFiles "~{reference_fasta}" \
      --sjdbGTFfile "~{reference_gtf}" \
```

```
      --sjdbOverhang ~{sjdb_overhang} \
      --genomeSAindexNbases ~{genome_sa_index_nbases}

    tar -czf star_index.tar.gz star_index/
  >>>

  output {                                                                ③
    File star_index_tar = "star_index.tar.gz"
  }

  runtime {                                                               ④
    docker: "getwilds/star:2.7.6a"
    memory: "~{memory_gb} GB"
    cpu: cpu_cores
  }
}
```

① Input for our task.
② Bash commands to execute in task
③ Description of output
④ Runtime requirements for execution. This is a lot like the `#SBATCH` directives.

Everything between the <<< and >>> is essentially a bash script. WDL has its own variables

16.10.3 Architecture of a WDL file

The best way to read WDL files is to read them top down. We'll focus on the basic sections of a WDL file before we see how they work together.

The code below is from the WILDs WDL Repo.

```
workflow SRA_STAR2Pass {
   meta{
   ...
   }

   parameter_meta{
   ...
   }

  input {                                                                 ①
    ...
```

```
  }

  if (!defined(reference_genome)) {                                    ②
    call download_reference {}
  }

  RefGenome ref_genome_final = select_first([reference_genome, download_reference.genome])

  call build_star_index {                                              ③
    ...
  }

  # Outputs that will be retained when execution is complete
  output {                                                             ④
    ...
  }

}
```

① Inputs for workflow
② If/then statement of workflow
③ call a task
④ Outputs of workflow

Let's go through each of these in detail (we'll get back to the `meta` and `parameters-meta` sections)

The structure of the workflow

```
workflow SRA_STAR2Pass {
  input {                                                              ①
    Array[SampleInfo] samples
    RefGenome? reference_genome
    String reference_level = ""
    String contrast = ""
  }

  if (!defined(reference_genome)) {                                    ②
    call download_reference {}
  }

  RefGenome ref_genome_final = select_first([reference_genome, download_reference.genome])
```

```
  call build_star_index { input:                                          ③
      reference_fasta = ref_genome_final.fasta,
      reference_gtf = ref_genome_final.gtf
  }

  # Outputs that will be retained when execution is complete
  output {                                                                ④
    ...
  }

}
```